

Scope affiliation front-end & backend + cancellation system

1.4 - Viator

I. Project and Document Objective and Key Assumptions of the Project

1. Rosotravel Website and System Rebuild
2. Viator API Integration and Product Data Handling
3. Viator Certification

II. Back-end – Changes for Viator Affiliate Products

1. Key Back-End Design Objectives for Affiliate Functionality (Viator Partner API)
 - a. One of the primary goals of the back-end design for affiliate functionalities is to meet all certification requirements for the Viator Partner API. - Viator API certification: back-end checks
 - b. The system must allow adding any Viator product related to our business activity, such as:
 - c. Compliance with the certification requirements.
2. Key Project Links and contact information
3. Changes to automatic location search in search and product surveys
 - a. scope includes:
 01. Frontend API (BFF – Backend for Frontend):
 - b. General rules (back-end)
 01. General rules
4. Booking Flow Between Rosotravel and Viator
 - a. General System Assumptions
- 3.1 Booking Modification Flow
 - a. General System Assumptions
 01. Role of Rosotravel
 02. Viator as Source of Truth
 03. Customer Communication Responsibility
 - b. Booking Modification Flow
 01. Objective
 02. Functional Scope
 03. Customer Request Registration
 04. Local Pre-Validation
 05. Sending Modification Request to Viator
 06. Viator Response Handling
 07. Additional Payment Handling
 08. Final Synchronisation of Booking State
 09. Successful Modification Outcome
 010. Failed Modification Outcome
 011. Exception Cases Requiring Operational Notification
 - c. Booking Cancellation Flow
 01. Objective
 02. Functional Scope
 03. Customer Cancellation Request

- 04. Cancellation Eligibility Check with Viator
- 05. Cancellation Request Submission to Viator
- 06. Final Cancellation Result Processing
- 07. Booking Status Update
- 08. Refund Data Storage and Downstream Processing
- 09. Customer Communication
- 010. Business Rule
- d. Customer Enquiry and Booking Clarification Flow
 - 01. Objective
 - 02. Functional Scope
 - 03. Local Resolution Scenario
 - 04. Viator / Supplier Escalation Scenario
 - 05. Enquiry Flow Requirements
 - 06. Conversation and Response Storage
- e. Contact Data Handling Between Rosotravel and Viator
 - 01. Objective
 - 02. Functional Scope
 - 03. Contact Data Requirements
 - 04. Email Routing Logic
 - 05. Business Use of Contact Configuration
 - 06. Recommended Default Configuration
- f. Responsibility Boundary
 - 01. Developer Responsibility
- 5. Category Mapping
 - a. General rules
 - 01. Categories (A–M) – TAXONOMIA
 - 02. Filters (facety) – (no categories)
 - 01. Weekly Automatic Category Update
 - 02. Uniqueness Requirement
 - b. Admin panel view: “Viator Categories”

Table with Category List

 - 01. Categories (A–M) – TAXONOMIA
 - 02. Filtry (facety) – (no categories)
 - c. Filters
 - 01. Categories (A–M) – TAXONOMIA
 - 02. Filters (facets) – (no categories)
 - d. Sorting

Sorting must be available by:
 - e. Manual Category Editing

Editing includes:

Validations:
 - f. Permissions
- 6. Billing and Settlement Principles
 - a. General rules

- 01. General Objective
 - 02. Simplified Initial Model
 - 03. Viator Invoice and Payment Logic
- b. Reporting File Generation
 - 01. Reporting Frequency
 - 02. Output Format
- 03. Email Distribution
 - 04. Scope of Reported Data
- c. Refunds, Chargebacks, and Disputes
 - 01. Manual Dispute Handling
 - 02. System Role in Dispute Support
 - 03. No Automatic Refund Settlement with Viator
- d. Finance Team Access and Verification
 - 01. Internal Finance Visibility
 - 02. Required Review Capability
 - 03. Simple Initial Interface
- e. Functional Direction of the Initial Solution
 - 01. Simple System Principle
 - 02. In Scope at This Stage
 - 03. Out of Scope at This Stage
- f. Operational Settlement Logic
 - 01. Viator Sends the Invoice
 - 02. Rosotravel Reviews and Pays the Invoice
 - 03. Reconciliation Support
- 7. Contract & Data/AI Architecture System
 - a. Workflow table
- 8. General rules for RAW Data Model + Unified Versioning
 - a. New destinations and raw product ingestion
 - 01. Destination implementation (Destination submit)
 - 02. Raw product ingestion
 - b. Raw Data Model (Unified Object Across All Entity Types)
 - 01. RAW Object (versioned snapshot)
 - 02. Hard rules
 - c. Execution: processing of products, seeds and the RAW engine INGESTION, VERSIONING & DIFFERENTIATION
 - d. Execution stage – general principles
 - 01. Entry points (how data reaches the execution stage)
 - 02. Addition of a new INTERNAL product
 - 03. Ingestion of a new EXTERNAL product
- 9. Table of raw versions for cities, attractions, products
 - a. RAW_CITY_Version (versioned snapshot)
 - 01. Core identity & snapshot meta
 - 02. content_fields (seed only; not indexing rules here)
 - 03. factual_fields

- 04. relationship_fields (MUST exist)
- 05. media_fields
- 06. hashes
- B. RAW_ATTRACTION_Version (versioned snapshot)
 - 01. Core identity & snapshot meta
 - 02. content_fields (seed only)
 - 03. factual_fields
 - 04. relationship_fields (MUST exist)
 - 05. media_fields
 - 06. hashes
- C. RAW_PRODUCT_Version (versioned snapshot + offer + real-time)
 - 01. Core identity & snapshot meta
 - 02. content_fields (seed)
 - 03. operational_fields (offer snapshot; medium cadence)
 - 04. realtime_offer_fields (high cadence; runtime cache)
 - 05. media_fields
 - 06. relationship_fields (MUST exist; includes internal white-label links)
 - 07. hashes
- D. Dedupe/Merge Rules (CITY & ATTRACTION)
 - 01. City dedupe keys (priority)
 - 02. Attraction dedupe keys (priority)
- E. Merge precedence (field winners) — RAW level
- F. Hard Requirements Checklist (developer enforcement)
- 10. Preupload Destinations — Functional Specification
 - a. Viator destination types
 - 01. List of types returned by Viator
 - b. General rules
 - 01. Types of destinations we want to include on our website
 - 02. Own ID (primary key)
 - 03. Viator External ID (optional)
 - 01. What we pre-upload
 - d. Destination normalisation and hierarchy mapping
 - f. Mapping strategy to the Rosotravel model
 - g. Data model
 - 01. Destination (Rosotravel)
 - 02. DestinationMapping (Viator)
 - h. System behaviour when changes are made on the Viator side
 - 01. Deletion of a destination by Viator
 - 02. Changes to the structure or types of destinations
 - i. To be confirmed with the developer during the project
- 11. Changes regarding automatic location search in the search engine and product surveys
 - a. Data Layer vs Frontend API
 - 01. Core Data Platform (integration backend – Viator Layer)

- b. General principles and purpose
 - 01. General principles
 - 02. Objective
 - c. Automatic location collection
 - 01. General description
 - 02. Scope
 - 03. Expected result
 - d. Recommended products
- 12. Flow for bookings between Rosotravel and Viator
 - a. Customer booking
 - 01. Purpose of the flow system
 - 02. Flow
 - b. Cancellation of a trip by the customer
 - 01. Purpose of the flow system
 - 02. Flow
 - c. Scope: Viator Supplier Communications (CLC) + CC to Rosotravel
 - 1) Objective
 - 2) Scope
 - 3) Integration requirement (request body)
 - 4) Configuration (ENV/Config)
 - 5) Validations (before sending to Viator)
 - 6) Payload example (fragment)
 - 7) Acceptance criteria
- 13. View all trips (Manage Products)
 - a. We want to add an additional filter to the view of all trips
 - 01. Filter by origin
 - 02. Information about displaying the page
- 14. Adding a trip (Create Product) - General rules
 - a. Selecting the trip type (product category)
 - 01. Change in nomenclature:
 - 02. Changes to the list layout
 - 03. How the list works.
- 15. Adding a Viator/TripAdvisor affiliate tour (Create Product)
 - a. General rules for adding affiliate tours
 - b. Tour type selection (product category)
 - c. Tour title and product code (basic information title)
 - 01. General rules:
 - d. Description and highlights (basic information description & highlights) - Affiliations
 - e. Itinerary - Viator Affiliations
 - 01. Itinerary view diagram
 - f. Location (city, country) and attractions (location) - Affiliations
 - 01. City and country categories:
 - 02. Attraction categories - Viator:
 - g. Tags, sections (keywords) - Affiliates

- h. Included in the price and not included in the price (Inclusions) - Affiliations
- i. Cancellation Policy - Affiliates
- j. Important information (additional information) - Affiliates
- k. Photos - Affiliates
- l. Options - Affiliates
- m. Meeting & collection point - Affiliates
- n. Pricing - Affiliates
 - 01. General rule for price retrieval
 - 02. Real-time price update (required by Viator)
 Perform an availability check for:
 - 03. Cache:
 - 04. Currency conversion
- r. Availability - Affiliates
 - 01. Expected result:
 - 02. Back-end operating principles
 - 03. Front-end operating principles
 - 04. Error handling
- s. Changes for Facility Duration
 - 01. Duration
- t. Administrator's opinions - new view - Affiliations
- u. Model for updating products and supporting data for Viator products - Affiliates
 - 01. Model assumptions
 - 02. Overall solution architecture
 - 03. Start of integration — Initial ingestion (first full feed)
 - 04. Cyclical updates — Delta updates (polling)
 - 05. Periodic full refresh of dictionary data (detection of changes and deletions)
 - 06. On-demand updates (when a new reference appears)
 - 07. Rate limiting and integration resilience
 - 08. Processing logic — data quality rules
- v. Definition of "done" (acceptance)
- 16. Single trip management panel - Affiliations
- 17. Changes in the view of all bookings
 - a. Additional filtering functions
- 18. Changes to the view of all ratings by affiliation
 - a. Additional filtering functions
 - b. Change to the rejected button
- 19. Flow for transactions detected as fraudulent (This is Viator's internal system, not ours)
- 20. Booking Change System on the Website (Rosotravel) (Back-end and Front-end section)
 - a. Context and Goal
 - 01. General Objective
 - 02. Product Type Distinction
 - 03. Shared and Separate Logic

- b. Scope of Customer Changes
 - 01. Parameters the Customer Can Change
 - 02. Parameter the Customer Cannot Change
 - 03. Rule for Option Changes
- c. Change Entry Points and UX Requirements
 - 01. General UX Requirement
 - 02. Dedicated Booking Page
 - 03. Customer Account – My Bookings
 - 04. UX Consistency with Cancellation Flow
- d. Common System Rules Applicable to Both Rosotravel and Viator Products
 - 01. Master Validation Rule
 - 02. Booking Versioning
 - 02.1. Scope of Stored Data
 - 02.2. History and Audit Trail
 - 02.3. Voucher Regeneration
 - 03. Financial Rules After a Change
 - 04. Shared Module for Participant Count Change Calculation
 - 04.1. Group Tours
 - 04.2. Private Tours
 - 05. Notification Rule
- e. Booking Change Flow for Rosotravel Own Products
 - 01. General Rule
 - 02. Customer Request Submission
 - 03. Blocking Option Change Requests
 - 04. Mandatory Validation
 - 05. Result of Negative Validation
 - 06. Availability Check
 - 07. Price Difference Calculation
 - 08. Surcharge, Refund, or Zero Difference
 - 09. Finalization of the Change
 - 010. Notifications on Success
 - 011. Cases Where bookings@rosotravel.com Must Not Be Notified
- f. Booking Change Flow for Viator Affiliate Products
 - 01. General Rule
 - 02. Customer Request Submission
 - 03. Mandatory Validation
 - 04. Sending the Change Request to Viator
 - 05. Viator Response Handling
 - 06. Free Change or Surcharge
 - 06.1. Participant Count Calculation Logic
 - 06.2. Binding Viator Amount
 - 07. Supplier Step
 - 08. Fetching Final Booking State
 - 09. Notifications on Success

- 10. Notifications on Failure
 - 10.1. No Notification for Systemically Impossible Changes
 - 10.2. Exception Cases Requiring Notification

IV. Front-end - change for Viator products

- 1. Front-End Design Changes
 - 01. General Front-End Design Direction
 - 02. Front-End Design Changes
 - 03. Nature of the Planned Changes
 - 04. Removal of the Previous Shop Page Model
 - 05. Purchase Flow Integration Within Target Pages
- 2. General Front-End Information
 - 01. Overall Goals for Front-End
 - 02. Compliance with Partner API Requirements
 - 03. Meeting Viator Certification Requirements
 - 04. Completion Criteria
- 3. Booking Confirmation Types – Global Concept
 - 01. General Application
 - 02. Instant Confirmation
 - 03. Manual Confirmation
 - 04. Instant-Then-Manual Confirmation
 - 05. Front-End Display Requirement
- 4. Changes to Facilities, Labels, Tags, and Filters for Viator Affiliate Products
 - a. General Principle
 - 01. Scope of Changes
 - 02. Category Section
 - 03. Implementation Principle
 - b. Elements Requiring Full Revision
 - 01. Facilities for Viator Affiliate Products
 - 02. Labels and Front-End Product Marking
 - 03. Tag Logic
 - 04. Filter Logic
 - c. Data Areas Covered by the Revision
 - 01. Duration
 - 02. Tour Type
 - 03. Booking-Related Labeling
 - 04. Guide Type
 - 05. Tour Vehicle
 - 06. Skip The Line / Priority Access
 - d. Monitoring and Processing Logic
 - 01. Processing Logs and Monitoring
 - 02. Scope of Revision
 - e. Final Scope Note
 - 01. Summary of What Must Be Changed
 - 02. Design Basis

4. Global Payment Behavior
 01. General Rule
 02. Instant Confirmation Payment Behavior
 03. Manual Confirmation Payment Behavior
 04. Instant-Then-Manual Payment Behavior
5. Global Rules for Partner Content
 01. General Principle
 02. Clear Attribution for Partner Content
 03. Explicit Partner Name Visibility
 04. Front-End Consistency Requirement
6. Pricing and Availability Rules for Affiliate Products
 - a. General Pricing Principle
 01. Default Display Price
 02. Fallback Price Category Order
 03. Purpose of the Default Price Logic
 - b. Pricing Rules Across the Website
 01. General Rule
 02. Listing Pages
 03. Product Page Before Date Selection
 04. Cart Pricing Rules
 05. Real-Time Price Update Before Payment
 06. Final Price Used for Booking Submission
 - c. Back-End Availability Rules for Affiliate Products
 01. Product Type Identification
 02. Availability Rules for Affiliate Products
 03. Admin Panel Rules
 04. Single Source of Truth
 05. Real-Time Availability Verification
 06. Cache Rules
 - d. Tour Page Application
 01. Scope of Application
7. Searches
 01. Overall goals for front-end
 1. Search capabilities
 - a. Behaviour based on input type
 - b. Handling typos and fuzzy searching
 8. Tour page / Product detail
 - a. New booking types & facilities icons
 - b. Reviews for affiliate products (This section can be changed by SEO & AI Scope)
 - c. Images (supplier & customer)
 - 8.1 Availability section ("Check Availability")
 - a. Real-time availability validation (Viator API) – REQUIRED
 - b. Age Categories

New age category: Senior

- c. Language Options — With Type Labels
 - User explanation tooltip
- d. Tour Option Display (Variants & Guide Information)
- e. Itinerary (Viator products only)
 - 01. Purpose
 - 02. Display rule
 - 03. Placement on the product page
 - 04. Itinerary content
 - 05. Front-end logic
- f. Cancellation policy & voucher type
- g. Tour operator / supplier information
- h. Pricing and Availability Rules for Affiliate Products
- 9. Changes to Discount Codes with Affiliation Information
 - a. Selectable List for the Discount Codes Section
 - 01. Product Scope Dropdown
 - 02. Purpose of the Field
 - b. Discount Code Logic in the Cart
 - 01. Product Source Verification
 - 02. Discount Application Rules
 - 03. Products Outside the Scope of the Code
 - c. Cart Communication to the User
 - 01. Applied Code Information
 - 02. Message Format
 - 03. Display Rule
- 10. Cart page
 - a. Main Purpose of the Cart
 - b. Display of Tours in Cart
 - 01. Expandable detail sections (per tour)
 - 02. Completed tours
 - c. CTA Behavior Based on Missing Data
 - d. Persistence of User Input
- e. Booking Questions — Full Integration with Viator API
 - 01. Types of booking questions the system must support
 - 02. Leader Customer Rules (Mandatory)
- f. Pickup, Meeting Point, End Point (Viator-compliant)
 - 01. If the tour supports PICKUP
 - 01.1 Provide pickup now
 - 01.2. Provide later
 - 01.3. Meet at departure point
 - 02. If the tour DOES NOT support pickup
 - 03. END POINT (tour may require a drop-off location)
- g. NEW MANDATORY RULE (Viator-flexible booking questions)
 - If Viator allows skipping a field → our system must also allow skipping it
- h. Additional Tour Information Requirements

- 01. Guide Type
 - i. Cancellation Policy Section (Rosotravel ONLY)
 - 01. Visibility rule (VERY IMPORTANT)
- 11. Payment page & Post-booking
 - a. Final summary & actions
 - b. Post-checkout communication (Viator requirements)

Functionalities – Viator Affiliate Products

I. Project and Document Objective and Key Assumptions of the Project

The purpose of this document is to define the scope of work required to rebuild the Rosotravel website and implement the Viator Merchant Partner integration within the Rosotravel system.

This project is not limited to displaying or copying Viator API data. The scope includes a system-level rebuild of the relevant Rosotravel components, including front-end, back-end, booking-related flows, product display logic, customer-facing processes, and internal operational logic.

All links in short:

I have generated a sandbox API key for your account, you can find it in your [Viator Partner Dashboard](#).

Here is the API documentation: <https://docs.viator.com/partner-api/technical/>.

We recommend checking our [Technical guide](#) as well as other helpful implementation articles in our Partner Resource Center:

- [Front-end guide for API Partners](#)
- [Certification documentation](#)
- [Managing product and availability data](#)
- [Implementing age bands & pax mix](#)
- [Calculating Product Pricing with the Merchant API](#)
- [Viator tags, explained](#)
- [Cancellation Policies and processes](#)
- [Automating supplier cancellations in V.2 Partner API](#)
- [Implementing Booking Questions with Viator's Merchant API](#)

I would like to schedule a kick-off call with your technical team to discuss your plans for this integration and answer any technical questions you might have. I will also explain in more detail our [certification requirements](#).

The project has three main objectives:

1. Rosotravel Website and System Rebuild

The first objective is to rebuild the Rosotravel website, <https://www.rosotravel.com>, into a new version with global changes across the website and system. This includes changes to the visual design, page structure, user experience, front-end components, back-end logic, and the core system architecture required to support the new version of the website.

The new Rosotravel system and the Viator integration must be designed and configured in such a way that Rosotravel can add and support any Viator service that is available for retrieval through the Viator Partner API and falls within the business scope accepted by Rosotravel.

The system must also support all relevant Viator booking methods and booking flows required for the products offered through the integration, including the booking behaviours and customer-facing requirements described in:

- [Implementation Guides for API Partners](#)
- [Viator APi partner](#).

This part of the scope applies to both:

- front-end development
- back-end development

2. Viator API Integration and Product Data Handling

The second objective is to build the front-end and back-end functionality required to retrieve product data from Viator through the Viator Partner API and make this data available within the Rosotravel system.

The integration must allow Viator product data to be received, stored, synchronised, and used by the Rosotravel website in accordance with the required system logic.

The rules related to processing, transforming, enriching, approving, and publishing Viator data are described in a separate scope document.

All functionalities implemented within the front-end scope must be designed and developed in accordance with the requirements and guidelines of the Viator Partner API.

This applies in particular to all customer-facing booking-related elements, partner content presentation, status communication, and user flow logic connected with Viator products. - [Viator Partner AP](#)

3. Viator Certification

The third objective is to meet all required Viator certification conditions for the integration.

All relevant back-end and front-end functionalities must be implemented in accordance with Viator Partner API requirements and certification criteria. The developer is responsible for ensuring that the implemented solution can be tested, demonstrated, documented, and confirmed by a Viator representative.

The front-end and back-end parts of the Viator integration shall be considered complete only after the relevant Viator certification requirements have been successfully fulfilled and confirmed.

➤ [Viator API Certification - Viator Partner Resource Center](#)

The developer is responsible for completing the following certification materials in consultation with the Viator representative:

- back-end certification form
- front-end certification form

All relevant functionalities covered by the Viator certification process must be implemented, tested, documented, and described in the certification forms in accordance with Viator's requirements and expected partner behaviour.

The developer's responsibility includes preparing the implemented solution for Viator's verification process, providing the required technical information, supporting the certification review, and making any necessary corrections if Viator identifies issues during verification.

The front-end and back-end parts of the Viator integration shall be considered complete only after the relevant certification forms have been submitted and the Viator representative has issued a positive certification verification result.

II. Back-end – Changes for Viator Affiliate Products

1. Key Back-End Design Objectives for Affiliate Functionality (Viator Partner API)
 - a. One of the primary goals of the back-end design for affiliate functionalities is to meet all certification requirements for the Viator Partner API. - [Viator API certification: back-end checks](#)
 - In the case of the developer, the main certification points that will need to be fulfilled are as follows:
 - Traveler cancellations
 - Supplier cancellations and amendments
 - Traveler-initiated amendments
 - Manual confirmation type products
 - b. The system must allow adding any Viator product related to our business activity, such as:
 - tickets
 - guided tours, including all types of services such as tastings, SPA trips, mountain trips, bike tours, etc.
 - transportation services
 - other services lasting up to 24 hours, carried out with guides, drivers, events or tickets, including those where the customer completes the experience independently, without a host
 - We exclude only tourism services not directly related to our operational activity, such as flight bookings, car rentals and hotel reservations.
 - c. Compliance with the certification requirements.
 - All back-end work should be carried out in a way that fully aligns with the criteria described in Viator's certification documentation.
 - The back-end work will be considered complete once Viator's certification document has been fully filled out, reviewed, and approved by a Viator representative. This means that both the technical implementation and the required certification tests must be successfully completed.
 - If any certification items are excluded from the project scope, the decision to omit them will be agreed upon and approved during the course of the project.

2. Key Project Links and contact information

The section below outlines the key project resources that should be treated as primary reference materials throughout the implementation process. The main objective of the project is to ensure full compliance with Viator certification requirements, covering both back-end and front-end implementation.

The API documentation is available at: <https://docs.viator.com/partner-api/technical/>.

In addition, the [Technical guide](#) and the following implementation resources available in the Partner Resource Center should also be reviewed:

- [Front-end guide for API Partners](#)
- [Certification documentation](#)
- [Managing product and availability data](#)
- [Implementing age bands & pax mix](#)
- [Calculating Product Pricing with the Merchant API](#)
- [Viator tags, explained](#)
- [Cancellation Policies and processes](#)
- [Automating supplier cancellations in V.2 Partner API](#)
- [Implementing Booking Questions with Viator's Merchant API](#)

3. Changes to automatic location search in search and product surveys

a. scope includes:

01. Frontend API (BFF – Backend for Frontend):

- Creating endpoints optimized for frontend needs (search, autocomplete, listings, product/detail cards).
- Aggregating data coming from the Data Platform layer.
- Formatting responses for the UI layer.
- Implementing caching to improve application performance.
- Presentation-layer logic and frontend integration.

02. Frontend

- Implementation of user interfaces.
- Development of components for search, forms, listings, and product cards.
- Handling UX flows and user interactions.

b. General rules (back-end)

01. General rules

- Use Rosotravel's internal Locations and Destinations database as the source of truth
- One integrated locations dataset used across the whole system:
 - Developer must ensure that the same locations dataset (countries, cities, and other location records available in the Rosotravel database) is used by:
 - the tours search engine (autocomplete and filtering by country/city),
 - checkout and operational forms (including pickup and meeting point fields) — searching and selecting locations must be performed against the same integrated Rosotravel database.
- Retrieve product locations and destinations from the Viator database. The back-end system developed by the developer must retrieve product locations and destinations from the Viator database created within the Rosotravel system, and then use this data across search, filtering, checkout, and forms.
- Destination timezone required and used in system time-based logic:
 - The developer must ensure that every destinations used in booking-related processes has an associated timezone retrieved from the Viator internal database, and that this timezone is used in time-based system logic — especially within the cancellation system (for example, calculating cancellation windows and cut-off moments in the destination's local time).

4. Booking Flow Between Rosotravel and Viator

a. General System Assumptions

For all Viator-sourced products sold on Rosotravel, the Rosotravel system shall act as the customer-facing system and the primary operational layer.

All customer actions related to a Viator booking, including:

- booking creation,
- booking modification,
- booking cancellation,
- clarification of booking details,
- operational customer enquiries,

shall first be recorded in the Rosotravel system and then synchronised with Viator where required.

Viator shall remain the source of truth for:

- final booking availability,

- modification eligibility,
- cancellation eligibility,
- refund value,
- supplier confirmation,
- supplier rejection,
- supplier-provided alternatives,
- final booking status related to the Viator product.

The Rosotravel system shall regularly retrieve updated booking data and statuses from Viator in order to ensure that the customer always sees the current booking state in Rosotravel.

Rosotravel, as the merchant, shall be responsible for all customer-facing communication related to Viator bookings, including:

- booking confirmations,
- booking modification confirmations,
- booking cancellation confirmations,
- refund communication,
- operational updates following supplier or Viator responses.

Viator may provide booking updates through API responses and, where applicable, through email notifications sent to Rosotravel, but the final communication to the customer shall always be sent by the Rosotravel system.

3.1 Booking Modification Flow

a. General System Assumptions

01. Role of Rosotravel

For all Viator-sourced products sold on Rosotravel, the Rosotravel system shall act as the customer-facing system and the primary operational layer.

All customer actions related to a Viator booking, including:

- booking creation,
- booking modification,
- booking cancellation,
- clarification of booking details,
- operational customer enquiries,

shall first be recorded in the Rosotravel system and then synchronised with Viator where required.

02. Viator as Source of Truth

Viator shall remain the source of truth for:

- final booking availability,
- modification eligibility,
- cancellation eligibility,
- refund value,
- supplier confirmation,
- supplier rejection,
- supplier-provided alternatives,
- final booking status related to the Viator product.

The Rosotravel system shall regularly retrieve updated booking data and statuses from Viator in order to ensure that the customer always sees the current booking state in Rosotravel.

03. Customer Communication Responsibility

Rosotravel, as the merchant, shall be responsible for all customer-facing communication related to Viator bookings, including:

- booking confirmations,
- booking modification confirmations,
- booking cancellation confirmations,
- refund communication,
- operational updates following supplier or Viator responses.

Viator may provide booking updates through API responses and, where applicable, through email notifications sent to Rosotravel, but the final communication to the customer shall always be sent by the Rosotravel system.

b. Booking Modification Flow

01. Objective

The purpose of this flow is to enable Rosotravel to handle customer-requested changes to Viator bookings while ensuring that:

- the modification request is properly recorded in Rosotravel,
- the modification request is verified and processed through Viator,
- the final booking state is retrieved from Viator,
- the updated booking data is stored in Rosotravel,
- all customer communication is handled by Rosotravel,
- required internal operational notifications are generated where applicable.

02. Functional Scope

The developer scope shall include implementation of the booking modification flow for Viator bookings within the Rosotravel system.

The modification flow shall support customer requests concerning, at minimum:

- date,
- time,
- option or variant,
- number of participants,
- pickup details,
- traveller details.

03. Customer Request Registration

The Rosotravel system shall allow the customer to submit a booking modification request from the Rosotravel interface.

The system shall store the requested modification in the Rosotravel database before further processing.

04. Local Pre-Validation

Before sending the request to Viator, the Rosotravel system shall perform a local pre-validation based on data already available in Rosotravel, including, where available:

- cancellation and modification policy,
- booking confirmation type,
- current booking status,
- known modification restrictions.

This local validation shall be treated as preliminary only.

The final decision regarding whether the modification is allowed, and under what conditions, shall always be based on the response returned by Viator.

05. Sending Modification Request to Viator

The Rosotravel system shall send the modification request to Viator with all required booking identifiers and all changed booking parameters.

The request payload shall include, where applicable:

- Rosotravel booking reference,
- Viator booking reference,

- changed date,
- changed time,
- changed option,
- changed number of participants,
- changed pickup details,
- changed traveller details,
- any other fields required by Viator for modification handling.

06. Viator Response Handling

The Rosotravel system shall be able to receive and process Viator responses indicating at minimum:

- modification allowed with no additional charge,
- modification allowed with additional charge,
- modification not allowed,
- manual follow-up required,
- supplier rejection,
- alternative proposal returned by Viator or supplier.

07. Additional Payment Handling

If Viator confirms that the requested modification is possible but requires an additional payment, the Rosotravel system shall:

- present the additional payment amount to the customer,
- block final modification confirmation until the payment is completed,
- proceed with final confirmation to Viator only after successful payment.

If the modification does not require any additional payment, the Rosotravel system shall proceed with the confirmation flow immediately.

08. Final Synchronisation of Booking State

After the modification process has been completed on the Viator side, the Rosotravel system shall retrieve the updated booking state from Viator.

The updated booking data shall include, where applicable:

- updated date,
- updated time,
- updated option,
- updated number of participants,
- updated pickup details,
- updated traveller details,
- updated booking price,

- updated booking status.

The updated data shall be stored in Rosotravel as the current valid booking version.

09. Successful Modification Outcome

If the booking modification process is completed successfully, the Rosotravel system shall:

- generate a new voucher with the updated booking data,
- send the updated voucher to the customer by email,
- update the booking status and booking details in the customer account area,
- store the final modification result in the Rosotravel system.

Additionally, the system shall automatically send an operational notification with the updated voucher to: **bookings@rosotravel.com**

This notification shall be sent only when the booking modification has been successfully completed.

010. Failed Modification Outcome

If the modification process is unsuccessful because the requested change is systemically impossible, the Rosotravel system shall:

- present the failed result to the customer,
- store the failed result in the booking history,
- not send any notification to **bookings@rosotravel.com**.

011. Exception Cases Requiring Operational Notification

Even if the modification process is not completed successfully, the Rosotravel system shall send a notification to:

bookings@rosotravel.com

in the following cases:

- the supplier actively rejected the request and the rejection was not automatic,
- Viator returned an alternative proposal,
- the process ended with a status equivalent to **manual follow-up required**,
- the customer did not confirm the required additional payment or requested change within the defined time limit.

c. Booking Cancellation Flow

01. Objective

The purpose of this flow is to enable cancellation of Viator bookings through Rosotravel while ensuring that:

- cancellation eligibility is verified through Viator,
- refund information is obtained from Viator,
- cancellation is recorded in Rosotravel,
- refund handling is based on Viator response data,
- all customer-facing communication is sent by Rosotravel.

02. Functional Scope

The developer scope shall include implementation of the booking cancellation flow for Viator bookings within the Rosotravel system.

03. Customer Cancellation Request

The Rosotravel system shall allow the customer to initiate a booking cancellation request from the Rosotravel interface.

The system shall allow the customer to select a cancellation reason from a predefined list maintained in Rosotravel.

04. Cancellation Eligibility Check with Viator

Before executing the cancellation, the Rosotravel system shall request from Viator the current cancellation conditions for the booking.

The system shall be able to process a Viator response containing, at minimum:

- whether cancellation is allowed,
- refund amount,
- refund currency,
- zero-refund result where applicable.

The Rosotravel system shall present this information to the customer before final cancellation submission, unless a separate business decision introduces a one-click cancellation model.

05. Cancellation Request Submission to Viator

If the customer accepts the cancellation conditions, the Rosotravel system shall send the cancellation request to Viator.

The request shall include the cancellation reason selected by the customer if required by Viator.

06. Final Cancellation Result Processing

The Rosotravel system shall retrieve and store the final cancellation result returned by Viator.

The system shall be able to handle at minimum:

- cancellation accepted,
- cancellation rejected,
- no refund,
- partial refund,
- full refund.

07. Booking Status Update

If cancellation is completed successfully through Viator, the Rosotravel booking record shall be updated accordingly.

The final booking status should clearly indicate that the booking was cancelled by the customer through the Viator flow.

08. Refund Data Storage and Downstream Processing

The Rosotravel system shall store the refund amount and refund currency returned by Viator.

This data shall be available for downstream payment, accounting, and finance-related processes handled by Rosotravel.

09. Customer Communication

After the cancellation process is completed, the Rosotravel system shall send the cancellation confirmation to the customer.

The communication shall include, at minimum:

- booking reference,
- Viator booking reference where available,
- cancellation result,
- refund amount,
- refund currency.

010. Business Rule

The cancellation confirmation and refund communication shall always be sent by the Rosotravel system, based on the data returned by Viator.

d. Customer Enquiry and Booking Clarification Flow

01. Objective

The purpose of this flow is to allow Rosotravel to handle customer operational enquiries related to Viator bookings, including both:

- enquiries that can be resolved using data already available in Rosotravel,
- enquiries that must be forwarded to Viator and/or the supplier.

02. Functional Scope

The developer scope shall include support for registering, processing, and storing customer enquiries related to Viator bookings.

03. Local Resolution Scenario

If the customer enquiry can be resolved using data already available in Rosotravel, the Rosotravel system shall allow internal handling without contacting Viator.

Examples include:

- changing pickup to another location already available on the approved pickup list,
- selecting a valid value already included in the allowed booking parameters,
- confirming operational details already stored in the booking or product content.

If the enquiry results only in selection of an already approved parameter, the Rosotravel system shall update the booking accordingly without requiring supplier approval.

04. Viator / Supplier Escalation Scenario

If the customer enquiry cannot be resolved using data available in Rosotravel, the Rosotravel system shall support forwarding the enquiry to Viator for supplier handling.

Examples include:

- pickup outside the supported city area,
- pickup from an airport or location not listed in the allowed pickup options,
- request for an additional passenger beyond standard booking rules,
- request for a time not included in the available time slots,
- any other operational clarification requiring supplier approval.

05. Enquiry Flow Requirements

The Rosotravel system shall:

- register the enquiry under the booking,
- determine whether the enquiry can be resolved locally,
- forward unresolved enquiries to Viator where required,
- store incoming Viator and/or supplier responses under the booking,
- display the response in the Rosotravel operational context,
- make the response available for communication to the customer.

If the supplier response leads to a booking modification requiring price or parameter changes, the Rosotravel system shall redirect the process into the booking modification flow described above.

06. Conversation and Response Storage

If a supplier response is received through Viator, including responses delivered operationally by email or equivalent channels, the Rosotravel system shall support attaching and storing that response under the relevant booking record.

The Rosotravel system shall provide sufficient booking-level visibility so that the customer support team can review the full operational history of the enquiry.

e. Contact Data Handling Between Rosotravel and Viator

01. Objective

The purpose of this part of the scope is to ensure that proper contact data is passed from Rosotravel to Viator during booking creation so that supplier communication can be routed correctly.

02. Functional Scope

The developer scope shall include implementation of the contact data structure required for Viator bookings.

03. Contact Data Requirements

During booking creation for a Viator product, the Rosotravel system shall always provide contact data required for supplier communication.

At minimum, the booking payload shall include:

- customer phone number in international format,
- email data according to the communication policy defined by Rosotravel.

04. Email Routing Logic

The Rosotravel system shall support the following email-routing options in the Viator booking payload:

- customer email only,
- Rosotravel support email only,
- customer email together with Rosotravel support email in one field, separated according to Viator requirements.

This configuration shall determine whether supplier communication is sent:

- directly to the customer,
- directly to Rosotravel,
- to both the customer and Rosotravel.

05. Business Use of Contact Configuration

If Rosotravel wants the supplier to contact the customer directly, the customer email shall be included.

If Rosotravel wants to maintain full communication control, only the Rosotravel support email shall be provided.

If Rosotravel wants the supplier to be able to contact the customer directly while ensuring that Rosotravel has full visibility of the conversation, both email addresses shall be provided in the booking payload according to Viator rules.

06. Recommended Default Configuration

The recommended default configuration is to pass:

- the customer email address, and
- the Rosotravel support email address

within the same email field, separated in accordance with Viator requirements.

This approach ensures that:

- the supplier may contact the customer directly where needed,
- Rosotravel retains visibility of supplier-customer communication,
- the support team can monitor what has already been communicated regarding the booking.

f. Responsibility Boundary

01. Developer Responsibility

Within this scope, developer is responsible for implementing the Rosotravel-side logic required to:

- register customer actions,
- send requests to Viator,
- retrieve and process responses from Viator,
- update booking data in Rosotravel,

- support customer account visibility,
- support operational visibility for the Rosotravel team,
- trigger Rosotravel-owned customer communication and internal notifications.
- supplier-side processing,
- internal Viator workflows,
- supplier decision logic,
- management of the supplier communication process within Viator,
- creation of any functionality on the Viator side beyond correct integration with the available Viator interfaces and flows.

5. Category Mapping

According to the latest assignment, the developer's goal is to rebuild the entire category system within the Rosotravel service, including the elements listed below:

- creation of the Marketing tab,
- creation, within that tab, of a subtab enabling category management and category editing,
- inclusion of this element within the developer scope of work, with the assumption that the method by which categories will be assigned in the future should be established,
- adoption of the initially imposed category mapping as the starting point for further work.

It should be noted that the categories listed below may be changed at later stages of the project.

Currently, the section below describes only how the new categories and their mapping should be structured.

a. General rules

01. Categories (A–M) – TAXONOMIA

A) Tickets & Attractions:

- Tickets & Entry Passes
- Museum Tickets

- Attraction Passes / City Pass
- Observation Decks
- Theme Parks
- Water Parks
- Zoos & Aquariums
- Shows Tickets
- Sports Event Tickets

B) Tours & Sightseeing:

- City Tours with Transport
- Walking Tours
- Hop-On Hop-Off
- Panoramic / Sightseeing
- Night Tours
- Private Tours
- Small Group Tours
- Self-Guided Tours (audio/GPS)
- Photography Tours
- Architecture Tours
- Street Art Tours
- Literary Tours
- Pop-Culture / Movie / TV Tours

C) Day Trips & Multi-Day:

- Day Trips Outside the City
- Multi-Day Tours
- Rail Day Trips
- Shore Excursions
- Layover Tours
- Overnight Tours
- Road Trips

D) Culture & History:

- Museums & Exhibitions
- Historical Sites & Heritage
- Archaeology Tours
- WWII & 20th Century History
- Religious Tours
- Local Culture & Traditions
- Hidden Gems

E) Food & Drink:

- Food Tours
- Street Food Tours

- Wine Tours
- Beer & Brewery Tours
- Distillery / Spirits Tours
- Tastings
- Cooking Classes
- Mixology / Cocktail Experiences
- Market Tours
- Dining Experiences

F) Cruises & Water Experiences:

- Sightseeing Cruises
- Dinner Cruises
- Sunset Cruises
- Day Cruises
- Catamaran / Sailing Trips
- Boat Rentals / Charters
- Kayaking / Canoeing Tours
- Snorkeling
- Scuba Diving
- Dolphin & Whale Watching
- Fishing Charters
- Rafting / River Trips

G) Outdoor & Adventure:

- Hiking Tours
- Cycling Tours
- E-Bike Tours
- Ziplining
- ATV / Off-road / 4x4
- Buggy / Dune Bashing
- Rock Climbing / Via Ferrata
- Caving
- Paragliding / Hang-gliding
- Skydiving
- Bungee Jumping
- Hot Air Balloon Rides

H) Winter & Mountain:

- Ski & Snowboard Lessons
- Lift Tickets
- Snowmobile Tours
- Dog Sledding
- Snowshoeing
- Ice Climbing

- Glacier Walks

I) Air Experiences:

- Helicopter Tours
- Seaplane Tours
- Scenic Flights
- Flight Simulator / Flight Instruction

J) Wellness & Relax:

- Spas & Thermal Baths
- Hammam / Turkish Baths
- Massage & Wellness
- Yoga / Meditation Experiences

K) Classes & Workshops:

- Art Workshops
- Dance Classes
- Music Classes
- Language / Cultural Classes
- Sports Classes

L) Sport & Active:

- Sports Experiences
- Watersports
- Extreme Sports
- Motorsport Experiences

M) Transfers & Transport Services:

- Airport Transfers
- Private Transfers
- Port Transfers
- Rail Transfers
- Transportation Passes
- Car Services / Chauffeur

02. Filters (facety) – (no categories)

a) Kolekcje / promowanie:

- Bestsellers
- Top Attractions

b) For who:

- Family-Friendly
- Adults-Only
- Couples

- Solo
- Groups / Friends
- Bachelor/Bachelorette

c) Format:

- Private / Shared
- Small Group
- Skip-the-Line
- Wheelchair Accessible
- Kid-friendly

d) Duration:

- 1–2h
- 2–4h
- 4–8h
- Full day
- Multi-day

e) Level:

- Easy
- Moderate
- Challenging

f) Price / Positioning:

- Budget
- Mid
- Luxury

Mapping Operation Rules

- When a new category is received from Viator and it has not been assigned previously, the system must add this category to the database in our back-end.
- Within the category database, there should be a filter for unmapped categories. The user must then be able to manually connect our category with the corresponding Viator category.
- Whenever a new category is received, an email notification about the newly added Viator category should be sent. The final email address will be determined during the project. For the time being, please use: b2bsales@rosotravel.com.

01. Weekly Automatic Category Update

- The system performs a Viator category synchronization once per week.

➤ The process includes:

- retrieving the current list of categories from the Viator API,
- detecting new categories that do not yet exist in the **viator_categories** table,
- sending a notification about the creation of a new tour category to the Marketing team at **b2bsales@rosotravel.com**,
- saving the category with the following fields:
 - ID
 - name
 - assigned_category (a new category is added as “Other” by default)

➤ The system must not overwrite categories that have been manually edited by an administrator or by a user with access to the Categories section within the Marketing subtab.

02. Uniqueness Requirement

➤ The **ID** field must be unique in order to prevent duplicates.

03. Mapping of Existing Rosotravel Categories to the New Categories

The full list of tags, together with the expected mapping structure, can be found at the link below:

link - [📄 category_mapping_1row_per_old](#)

04. Mapping of Viator Categories to the New Rosotravel Categories

The full list of tags, together with the expected mapping structure, can be found at the link below:

link - [📄 viator_to_new_taxonomy_mapping](#)

The final structure of the website categories is still to be agreed with the Marketing team.

b. Admin panel view: “Viator Categories”

Table with Category List

Columns:

- **Category name** (string) – name of the Viator category
- **Assigned main category** (enum) – one value to be selected from the full list of categories (A–M) – **TAXONOMY**
 - [01. Categories \(A–M\) – TAXONOMIA](#)
- **Assigned additional category** – one value to be selected from the following:
 - [02. Filtry \(facet\) – \(no categories\)](#)
- **Created at** (datetime) – date when the category was added to the system
- **Updated at** (datetime) – date of the last assignment edit

- **Edit** – button allowing manual mapping of the Viator category to our category

c. Filters

At the top of the view, the following filters are required:

- **Filter by Assigned Main Category** (dropdown), values:
 - one value to be selected from the full list of categories (A–M) – **TAXONOMY**
 - [01. Categories \(A–M\) – TAXONOMIA](#)
- **Filter by additional category**, values:
 - [02. Filters \(facets\) – \(no categories\)](#)
- **Search bar** – search by Viator category name (partial match, case-insensitive)

d. Sorting

Sorting must be available by:

- category name **A → Z / Z → A**
- date added (**Created at**) **↑ / ↓**

Sorting must be handled on the back-end side.

e. Manual Category Editing

Each row must include an “**Edit**” button.

Editing includes:

- changing the **Assigned main category** field (enum)
- saving changes via: **PATCH /admin/viator-categories/{id}**

Validations:

- the value must be one of the allowed enum values
- saving the change must update the **updated_at** field

f. Permissions

Access must be limited to the following roles only:

- admin
- content manager

6. Billing and Settlement Principles

a. General rules

01. General Objective

The purpose of this part of the scope is to introduce a simple financial reporting and settlement support mechanism for Viator bookings within the Rosotravel system.

This mechanism is intended to support internal finance operations, provide visibility into booking-level financial data, and enable regular reporting to both Rosotravel and Viator in line with the commercial and invoicing logic set out in the Viator agreement.

02. Simplified Initial Model

At this stage, the system shall be implemented as a simple operational and reporting solution.

The purpose of the solution is not to automate the full dispute, refund, chargeback, or payment settlement lifecycle with Viator, but rather to:

- collect the relevant reservation and financial data,
- make such data visible for internal review,
- generate structured reporting files,
- support the finance team in verifying and reconciling bookings,
- support manual settlement and dispute processes outside the system where required.

03. Viator Invoice and Payment Logic

Viator issues invoices to Rosotravel in accordance with the rules defined in the agreement.

Rosotravel does not issue settlement invoices to Viator under this process.

Instead:

- Viator sends its invoice to Rosotravel,
- Rosotravel is responsible for reviewing that invoice,
- Rosotravel is responsible for paying Viator in accordance with the invoice and the contractual payment deadlines.

The Rosotravel system shall therefore support internal verification and reconciliation of booking-level financial data, but shall not be treated as the formal issuer of the invoice payable to Viator.

b. Reporting File Generation

01. Reporting Frequency

The system shall support generation of a financial settlement report once every two weeks.

This report shall cover the applicable reporting period and shall include Viator-related reservations and their financial values required for internal review and external sharing.

02. Output Format

The report shall be generated in a format suitable for operational and finance use, specifically:

- Excel file, or
- Google Sheet compatible export format.

The final technical format may be defined during implementation, but the file must be easy to review, filter, and share by the finance team.

03. Email Distribution

Each reporting file shall be prepared for email-based distribution.

The report shall be sent by email to:

- finance@rosotravel.com
- Viator finance / settlement contact address, to be defined operationally

The mailing process may be implemented as:

- a simple automated email dispatch, or
- a semi-manual process supported by the system

depending on the final implementation decision.

04. Scope of Reported Data

The report shall include the core financial fields required for settlement verification under the Viator cooperation model.

The exact column set shall be finalised during implementation, but it is expected to include at minimum:

- reservation identifier,
- Viator booking reference,
- booking date,
- travel date or fulfilment date,
- retail price,
- net price,
- commission payable to Viator,
- commission rate,
- amount retained by Rosotravel,
- currency,
- booking status,
- cancellation or refund status where applicable.

If additional fields are required by the finance team or by Viator's settlement expectations, such fields may be added during further implementation stages.

c. Refunds, Chargebacks, and Disputes

01. Manual Dispute Handling

Refunds, chargebacks, and booking-related discrepancies that require formal financial challenge toward Viator shall not be processed automatically within this stage of the system.

If Rosotravel identifies a discrepancy, refund-related inconsistency, or chargeback issue, such matter shall be handled manually as a dispute process in accordance with the agreement.

02. System Role in Dispute Support

Although disputes will not be automated, the system shall support the dispute process by ensuring that the relevant reservation and financial data is visible and traceable.

The system shall therefore allow the finance team to review the relevant booking data, amounts, and statuses in order to prepare manual dispute submissions where needed.

03. No Automatic Refund Settlement with Viator

At this stage, the system shall not automatically:

- submit disputes to Viator,
- create financial adjustment requests toward Viator,
- process chargeback claims with Viator,
- trigger automatic settlement corrections toward Viator.

These actions shall remain outside the automated scope and shall be handled manually by Rosotravel.

d. Finance Team Access and Verification

01. Internal Finance Visibility

The finance team must have access to booking-level financial and operational data required to verify whether the financial settlement with Viator is consistent.

This access is necessary in particular for cases where:

- invoice values do not match internal expectations,
- booking values appear inconsistent,
- refunds or cancellations require review,
- a dispute may need to be raised.

02. Required Review Capability

The system shall provide a simple internal view allowing the finance team to check reservations and related financial data.

This view should allow at minimum:

- searching for a reservation,
- identifying whether the booking relates to Viator,
- reviewing booking references,
- reviewing booking status,
- reviewing financial values used for settlement,

- checking whether the booking appears in the relevant reporting period.

03. Simple Initial Interface

At this stage, the finance control area does not need to be implemented as an advanced accounting module.

A simple internal panel, table view, or reporting view is sufficient, provided that it gives the finance team the practical ability to review reservations and identify discrepancies.

e. Functional Direction of the Initial Solution

01. Simple System Principle

The overall solution should remain simple at the current stage.

The objective is to build a practical and usable internal finance-support mechanism rather than a fully automated accounting and settlement platform.

02. In Scope at This Stage

At this stage, the system should focus on:

- collecting reservation-level financial data,
- distinguishing Viator bookings from other bookings,
- generating regular settlement reports,
- supporting email-based report distribution,
- giving the finance team visibility into reservation records,
- supporting manual reconciliation with Viator invoices.

03. Out of Scope at This Stage

At this stage, the following elements should remain outside the automated scope:

- automated dispute submission,
- automated chargeback handling with Viator,
- automated refund claim management with Viator,
- full accounting reconciliation engine,

- complex finance workflow automation,
- automated invoice payment to Viator.

f. Operational Settlement Logic

01. Viator Sends the Invoice

Under the agreed operating model, Viator sends the invoice to Rosotravel.

The Rosotravel system is therefore not responsible for issuing the final settlement invoice toward Viator.

02. Rosotravel Reviews and Pays the Invoice

After receiving the invoice from Viator, Rosotravel reviews the invoice internally and pays it in accordance with the contractual terms.

The purpose of the Rosotravel system is to support this review process by making the relevant booking and financial data available in a structured and auditable format.

03. Reconciliation Support

The system shall help Rosotravel compare:

- reservations visible in Rosotravel,
- booking references linked to Viator,
- commercial values calculated internally,
- data included in the reporting file,
- amounts later invoiced by Viator.

This reconciliation support is intended to reduce the risk of discrepancies and to provide the finance team with the operational basis for manual follow-up if inconsistencies arise.

7. Contract & Data/AI Architecture System

a. Workflow table

Step	Owner	Tooling	Trigger	What it does	Output
1. Ingest raw feed	Developer AI	Viator API → DB	Schedule (hourly/daily)	Store raw vN JSON (draft_raw)	/raw/vN/{id}.json

2. Versioned diff	Developer BE	Diff engine (hash per section)	On new vN	Compare vN vs active_version; produce change_map	change_map.json
3. Routing	Developer BE	Policy rules	After diff	Classify: minor / medium / major	regen_queue tasks
4. Programmatic gen	Developer AI	OpenAI + LangChain	medium/major OR first publish	Create structured content (meta, H1–H3, summary, FAQ, tags, links, alt)	/drafts/{slug}.json
5. Validation	Developer BE	Schema linter + business rules	After gen	Check consistency (fields), schema.org validity	validated_draft.json
6. Uniqueness (EN only)	Developer + SEO QA	Copyscape (EN, ai_modified)	First publish + medium/major re-gen	External duplicate check; if fail → human fix → recheck	uniqueness_report.json
7. Human QA	Rosotravel Marketing	Internal panel	Post-validation	Spot-check, edits, approve	Approved draft
8. Publish	Developer FE	Publishing service + sitemap builder	On approval	Set published status, lock URL, set index/canonical/hreflang flags, update sitemaps/lastmod Output: Live URL + updated sitemaps	Live URL
9. Translate	Developer BE	Azure Translator/Chat GPT	Post-publish EN	Create DE/FR/IT/PL/ES + hreflang	Localised pages
10. Metrics loop	Developer BE	GA4 + GSC + GPT	Nightly	Flag low CTR/Bounce; suggest targeted re-gen	optimisation_queue
11. Near-pages	Developer AI	GSC (+Semrush) + embeddings + OpenAI	Weekly	Create near-page drafts for real demand (no cannibalisation)	New drafts

12. Internal similarity	SEO QA	Screaming Frog N-gram	Monthly	Check <15% overlap; flag templates/clusters	Similarity report
13. LLM feeds	Developer	ai-sitemap, ai-summary, feeds	Nightly	Refresh AI endpoints for ChatGPT/Bing/Perplexity	Fresh feeds

8. General rules for RAW Data Model + Unified Versioning

a. New destinations and raw product ingestion

01. Destination implementation (Destination submit)

The list of destinations is retrieved from the Viator API. The destinations are then saved in the Rosotravel database and mapped to the destination types we support (country, region, city). This mapping is crucial because the assigned destinations must be refreshed weekly. As a result of this process, a hierarchy (tree) of destinations is created.

02. Raw product ingestion

- All Viator affiliate products are retrieved via API and saved as RAW entries along with version numbers, hashes, and data source information.
- RAW entries are grouped into packages by country to enable scalable keyword assignment and clustering. For example, all products can be placed in one package, e.g. "Italy raw products"..
- The system stores RAW product data in draft form in English, without generating any SEO elements.
- During ingestion, a placeholder is created to supplement factual data, which will be used in the future to supplement metadata about attractions and cities (Google Places + Wikidata).

b. Raw Data Model (Unified Object Across All Entity Types)

As part of the RAW Data Model, the contractor is responsible for:

- Creating the technical structure of the RAW Data Model to the extent necessary to store data retrieved from the API.
- Basic Core elements on which the Raw model is based
- The model data should be in English

- Downloading destinations in the form of a pre-upload for countries and regions
- Updating the RAW model

Every entity entering the system—product, attraction (POI), city, region, country, category, bundle—MUST follow the same RAW ingestion model:

01. RAW Object (versioned snapshot)

```
RAW_Entity_Version {
  entity_id: internal_id,
  entity_type: PRODUCT | ATTRACTION | CITY | REGION | COUNTRY |
    CATEGORY | BUNDLE,
  external_id: viator_id | rosotravel_id | null,
  source_type: INTERNAL | EXTERNAL,

  raw_version: N,                // increments only on new source snapshot
  raw_ingested_at: timestamp,

  raw_payload: { ... },          // full unmodified source payload(s),
    internal-only

  // Normalised blocks derived from raw_payload (still RAW-owned):
  content_fields: {
    title, short_description, long_description,
    highlights[],
    itinerary_overview            // short, non-stepwise, Rosotravel-owned
    narrative
  },

  operational_fields: {
    price_from, currency, duration, availability_refs,
    cancellation policy, meeting point, languages, confirmation type,
    included[], excluded[],        // factual/operational offer components (not
    AI-authored)
    itinerary_steps_raw            // supplier/internal steps stored for UI only
    (restricted)
  },

  factual_fields: {
    opening_hours, address, coordinates, telephone, official_website,
    place_id, wikidata_id, sameAs[]
  },
}
```

```

media_fields: {
  images[], videos[]
},

hashes: {
  content_hash,
  operational_hash,
  factual_hash,
  media_hash
}
}

```

02. Hard rules

- RAW objects are stored unchanged, never replaced, always versioned.
- raw_payload is internal-only and MUST never be exposed in:
 - front-end responses,
 - public APIs,
 - any sitemap,
 - AI feeds,
 - MCP responses,
 - or any indexable HTML.
- Public pages and public feeds MUST use ONLY the Published Record (see below), assembled from permitted, licence-safe fields.

c. Execution: processing of products, seeds and the RAW engine INGESTION, VERSIONING & DIFFERENTIATION

Within the scope of the following developer, this only applies to what is relevant for creating basic core drafts. This information is mainly for the developer's reference.

The purpose of this system component is to ensure that:

- all input data (internal and external) enters the system securely
- each piece of data is versioned, hashed, classified and stored
- only those parts of the data that have actually changed are reprocessed (in order to reduce costs and maintain SEO stability)
- manual edits made by humans always take precedence over API updates
- content rewriting by AI is controlled by strict rules at the individual field level
- there is never any destructive overwriting of data
- the system can operate continuously, securely and scalably

This layer defines how **RAW** data becomes supervised, stable, and predictable input for the AI pipeline.

It forms the foundation for:

- keyword ownership management
- factual data enrichment
- AI-generated content outlines
- structural data generation (schema)
- publication
- tracking and monitoring

d. Execution stage – general principles

01. Entry points (how data reaches the execution stage)

- The Execution process begins only when at least one of the following events occurs:

02. Addition of a new INTERNAL product

- a new product is added manually / via CMS
- the publication team adds the product
- the following are assigned: country (or region) + city (must already exist in the system) and, optionally, an attraction is selected or created via Google Maps
- the product is saved as an INTERNAL source

03. Ingestion of a new EXTERNAL product

- the ingestion process retrieves product data from an external API (Viator / OTA)
- the destination chain comes from external destination identifiers
- the product is saved as an EXTERNAL source

Key principle: regardless of the entry point, each entity (Product / City / Attraction) MUST first enter the system as a RAW version (no direct publication).

9. Table of raw versions for cities, attractions, products

a. RAW_CITY_Version (versioned snapshot)

01. Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes
entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	CTY_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	CITY

source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL
external_id (viator_destination_id)	EXTERNAL API (OTA)	ONCE_ID	No	No	Required if source_type=EXTERNAL
google_place_id	GOOGLE	ONCE_ID	No	No	Identity key if available
wikidata_id (QID)	WIKIDATA	ONCE_ID	No	No	Identity key if available
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* "Required if EXTERNAL" = hard validation per source_type.

02.content_fields (seed only; not indexing rules here)

Field	Source Order	Refresh Policy	Required	Derived	Notes
name_canonical	GOOGLE → WIKIDATA	GP_IDENTITY_90D	Yes	No	Canonical display name
name_aliases[]	WIKIDATA	WD_ALIASES_180D	No	No	Alternative names
short_description_seed	WIKIDATA → (Wikipedia optional)	WD_CORE_365D	No	No	Seed only; can be empty initially

03.factual_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes
coordinates {lat,lng}	GOOGLE → WIKIDATA	GP_GEO_365D	Yes	No	Mandatory for geo linking

address_structured	GOOGLE	GP_IDENTITY_90D	No	Yes	Normalised from components
time_zone	EXTERNAL API (OTA) → GOOGLE	OTA_RELATIONS_180D	No	No	Prefer OTA if consistent
iata_codes[]	EXTERNAL API (OTA)	OTA_RELATIONS_180D	No	No	Optional
categories[]	GOOGLE → WIKIDATA	GP_CATEGORIES_180D	No	No	Place categories
official_website	GOOGLE → WIKIDATA	GP_CONTACT_60D	No	No	Optional

04.relationship_fields (MUST exist)

Field	Source	Refresh Policy	Required	Derived	Notes
parent_relations.country	OTA graph → Wikidata	OTA_RELATIONS_180D	Yes	Yes	Must exist as reference
parent_relations.region	OTA graph → Wikidata	OTA_RELATIONS_180D	No	Yes	Optional if not available
parent_relations.city (self)	SYSTEM	ONCE_ID	Yes	Yes	Self pointer
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	CITY_IN_REGION + REGION_IN_COUNTRY
relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

05. media_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes
images[]	GOOGLE → WIKIDATA → Wikipedia(opt)	GP_CATEGORIES_1 80D	No	No	References only; avoid frequent paid pulls
videos[]	GOOGLE (if any)	GP_CATEGORIES_1 80D	No	No	Optional

06. hashes

Field	Source	Required	Notes
content_hash	SYSTEM	Yes	Hash of City content_fields only (name + aliases + desc seed)
factual_hash	SYSTEM	Yes	Hash of factual_fields only
media_hash	SYSTEM	Yes	Hash of media_fields
relationship_hash	SYSTEM	Yes	Hash of relationship_fields

B. RAW_ATTRACTION_Version (versioned snapshot)

01. Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes
entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	POI_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	ATTRACTION
source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL

external_id (viator_attraction_id)	EXTERNAL API (OTA)	ONCE_ID	No	No	Required if EXTERNAL and provided
google_place_id	GOOGLE	ONCE_ID	No	No	Created via Google for internal creation
wikidata_id (QID)	WIKIDATA	ONCE_ID	No	No	Alternative identity key
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* One-of identity rule (hard): attraction must have at least one of (viator_attraction_id OR google_place_id OR wikidata_id).

02.content_fields (seed only)

Field	Source Order	Refresh Policy	Required	Derived	Notes
name_canonical	GOOGLE → WIKIDATA	GP_IDENTITY_90D	Yes	No	
introduction_seed	OTA unique content → Wikidata	OTA_PRODUCT_CONTENT_90D	No	No	RAW only; indexing handled elsewhere
overview_sections_seed[]	OTA unique content → Wikidata	OTA_PRODUCT_CONTENT_90D	No	No	RAW only

03.factual_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes
-------	--------------	----------------	----------	---------	-------

coordinates {lat,lng}	GOOGLE → WIKIDATA	GP_GEO_365D	Yes	No	
address_structured	GOOGLE	GP_IDENTITY_90D	No	Yes	
opening_hours	GOOGLE only	GP_HOURS_7D	No	No	Optional if unavailable
official_website	GOOGLE → WIKIDATA	GP_CONTACT_60D	No	No	
telephone	GOOGLE	GP_CONTACT_60D	No	No	
categories[]	GOOGLE → WIKIDATA	GP_CATEGORIES_180D	No	No	
sameAs[]	WIKIDATA	WD_CORE_365D	No	No	External IDs/links

04.relationship_fields (MUST exist)

Field	Source	Refresh Policy	Required	Derived	Notes
parent_relations.city	OTA destinations → destination_map OR GOOGLE address	OTA_RELATIONS_180D	Yes	Yes	Canonical city required
parent_relations.region	from City chain	Derived on compute	No	Yes	Optional
parent_relations.country	from City chain	Derived on compute	Yes	Yes	Derived via city parent
parent_relations.attraction (self)	SYSTEM	ONCE_ID	Yes	Yes	Self pointer
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	ATTRACTION_ID_N_CITY +

					inherited city chain
relationship_hash	SYSTEM	on change	Yes	Yes	

05.media_fields

Field	Source	Refresh Policy	Required	Notes
images[]	OTA → GOOGLE → WIKIDATA	OTA_MEDIA_30D (OTA part) + GP_CATEGORIES_180D (Google part)	No	Keep source attribution
videos[]	OTA (if any)	OTA_MEDIA_30D	No	Optional

06.hashes

Field	Required	Notes
content_hash	Yes	content_fields
factual_hash	Yes	factual_fields
media_hash	Yes	media_fields
relationship_hash	Yes	relationship_fields

C. RAW_PRODUCT_Version (versioned snapshot + offer + real-time)

01.Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes
entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	PRD_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	PRODUCT

source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL
external_id (viator_productCode)	EXTERNAL API (OTA)	ONCE_ID	No	No	Required if EXTERNAL
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version (content/media/relations changes)
offer_version	SYSTEM	on change	Yes	Yes	Medium cadence offer snapshot (operational offer)
realtime_offer_version	SYSTEM	frequent	Yes	Yes	High cadence pricing/availability
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* Required if source_type=EXTERNAL

02.content_fields (seed)

Field	Source Order	Refresh Policy	Required	Derived	Notes
title_seed	OTA unique content → OTA title	OTA_PRODUCT_CONTENT_90D	Yes	No	INTERNAL: CMS title_seed required
short_description_seed	OTA unique content → OTA description	OTA_PRODUCT_CONTENT_90D	No	No	Optional seed
long_description_seed	OTA unique content → OTA description	OTA_PRODUCT_CONTENT_90D	No	No	Stored RAW; indexing handled elsewhere

highlights_seed[]	OTA tags/de sc → SYSTEM extract	OTA_PRODUCT_CONT ENT_90D	No	Yes	Extract bullets seed if available
itinerary_overview_seed	OTA itinerar y → SYSTEM summa ry seed	OTA_PRODUCT_CONT ENT_90D	No	Yes	Seed only

03. operational_fields (offer snapshot; medium cadence)

Field	Source	Refresh Policy	Req uire d	Derive d	Notes
duration	EXTERNAL API (OTA)	OTA_STABLE_180D	Yes	No	
languages[]	EXTERNAL API (OTA)	OTA_STABLE_180D	No	No	
meeting_point	OTA logistics → (optional Google verify)	OTA_OFFER_30D	No	No	Verify only if needed (do not poll Google)
cancellation_po licy	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	
included[]	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Optional; warn if missing
excluded[]	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Optional; warn if missing
product_options []	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Offer structure
confirmation_ty pe	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	

04.realtime_offer_fields (high cadence; runtime cache)

Field	Source	Refresh Policy	Required	Derived	Notes
price_from	OTA pricing	REALTIME_15M	Yes	No	Must not bump raw_version
currency	OTA pricing	REALTIME_15M	Yes	No	
availability_refs	OTA availability	REALTIME_15M	Yes	No	Calendar endpoint references
start_times[]	OTA availability	REALTIME_15M	No	No	Optional; not always explicit

05.media_fields

Field	Source	Refresh Policy	Required	Notes
images[]	EXTERNAL API (OTA)	OTA_MEDIA_30D	No	
videos[]	EXTERNAL API (OTA)	OTA_MEDIA_30D	No	

06.relationship_fields (MUST exist; includes internal white-label links)

Field	Source	Refresh Policy	Required	Derived	Notes
parent_relations.city	INTERNAL: CMS selection / EXTERNAL: OTA destinations	OTA_RELATIONS_180D	Yes	Yes	Canonical city required
parent_relations.region	from City chain	Derived on compute	No	Yes	
parent_relations.country	from City chain	Derived on compute	Yes	Yes	

parent_relations. attraction	INTERNAL: CMS/Google selection / EXTERNAL: link if resolvable	OTA_RELAT IONS_180D	No	Yes	Optional but recommended
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	PRODUCT_IN_ CITY + optional PRODUCT_VIS ITS_ATTRACTI ON
product_attraction_link_code	SYSTEM	ONCE_ID	No	Yes	LNK_<ULID> created when link exists
relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

07. hashes

Field	Required	Notes
content_hash	Yes	content_fields only (NO real-time signals)
offer_hash	Yes	operational fields only
realtime_offer_hash	Yes	realtime_offer_fields only (NO raw_version bump)
media_hash	Yes	media_fields
relationship_hash	Yes	relationship_fields

D. Dedupe/Merge Rules (CITY & ATTRACTION)

01. City dedupe keys (priority)

Priority	Key	Source	Rule
1	viator_destination_id	VIATOR	If matches → same CITY
2	google_place_id	GOOGLE	If matches → same CITY
3	wikidata_id (QID)	WIKIDATA	If matches → same CITY

4	(name + coordinates within radius)	SYSTEM	Only if none above exist; requires manual review if collision
---	------------------------------------	--------	---

Hard rule: If two keys conflict (e.g., same google_place_id but different viator_destination_id), keep one canonical City and store the other as alias reference; raise conflict flag.

02. Attraction dedupe keys (priority)

Priority	Key	Source	Rule
1	viator_attraction_id	VIATOR	If matches → same ATTRACTION
2	Google Place ID	GOOGLE	If matches → same ATTRACTION
3	wikidata_id (QID)	WIKIDATA	If matches → same ATTRACTION
4	(name + coordinates within radius + same city)	SYSTEM	Only if none above exist; requires review

Hard rule: Attraction must always have canonical City relationship; merge cannot produce attraction without city.

E. Merge precedence (field winners) — RAW level

This is NOT publishing logic; it only unifies RAW entity identity.

Field group	Winner precedence
name/address/geo	GOOGLE first
hierarchy destination chain	VIATOR first
historical facts	WIKIDATA first
external IDs	WIKIDATA first
product commercial content	VIATOR first
offer/realtime offer	VIATOR first

F. Hard Requirements Checklist (developer enforcement)

1. Every CITY/ATTRACTION/PRODUCT must have:
 - entity_id, stable_code, raw_version, raw_ingested_at
 - relationship_fields + relationship_edges[] + relationship_hash
 - hashes required by entity type (see 6.4 matrix)
2. Every CITY/ATTRACTION/PRODUCT must store:
 - parent_relations (including country reference)
 - relationship_edges[]
 - relationship_hash
3. PRODUCT must always have:
 - offer_version + offer_hash
 - real-time offer version + real-time offer hash
 - hard separation: real-time never increments raw_version
4. INTERNAL products must mint stable codes immediately:
 - product stable_code PRD_<ULID>
 - attraction stable_code POI_<ULID> (if created)
 - link_code LNK_<ULID> if product ↔ attraction exists

This enables deterministic white-label exports.
5. The Viator hierarchy must be stored:
 - destinationId and parentDestinationId chain snapshot (via parent_relations + edges)

10. Preupload Destinations — Functional Specification

a. Viator destination types

01. List of types returned by Viator

- AREA
- CITY
- COUNTRY
- COUNTY
- DISTRICT
- HAMLET
- ISLAND
- NATIONAL PARK
- NEIGHBOURHOOD
- PENINSULA
- PROVINCE
- REGION
- STATE
- TOWN
- Union Territory
- VILLAGE

- WARD

b. General rules

01. Types of destinations we want to include on our website

- Country
- Region (new type of destination on the Rosotravel website)
- City

This list is treated as a primary source and is not created dynamically based on products. It is used to build a hierarchical structure such as: **country** → **region** → **city** (or, for example, **country** → **island**).

Key principle:

- When selecting a country for a product, it must also be automatically linked to the region and city according to **the country** → **region** → **city** principle.

02. Own ID (primary key)

- destination_id — Rosotravel internal ID
- The ID never changes
- Used in:
 - SEO
 - routing
 - listing
 - product assignment Rosotravel
- destination_type (country or region or city)
- parent_id (for regions)
- canonical_name in English
- external_id (e.g., Viator destinationId)

Important rule: during import, the system must create a base version (1:1 as on Viator) of the destination in the form of a RAW object for each record.

This RAW destination snapshot is later used exclusively for:

- retrieving keywords based on seeds
- generating destination pages (Run SEO Auto process).

03. Viator External ID (optional)

- viator_destination_id
- used solely for mapping affiliate products
- is not a source of truth

If Viator:

- deletes a destination,
- changes its status,

- changes its structure,
- then:
- the Rosotravel destination remains active,
 - Viator products associated with this destination are deactivated,
 - Rosotravel products remain visible and active.

c. Pre-upload scope

01. What we pre-upload

- COUNTRY
- REGION

02. What we do not pre-upload

- CITY (cities)

Cities:

- are created dynamically when adding trips,
- are not part of the pre-upload.

d. Destination normalisation and hierarchy mapping

After uploading, the system launches an automatic normalisation process, which:

- standardises destination names to a single canonical form in English,
- resolves duplicates (e.g. "UK" vs "United Kingdom"),
- builds superior–subordinate relationships between countries and regions.

No SEO content is generated at this stage.

e. Connection to products

When adding products (the product is the trigger for the process), the system connects destinations with all products using `external_id` / `destinationId` mapping from Viator and internal products.

For each destination, the system calculates the `product_count` and marks it as:

- active (has at least one product)
- inactive (no products assigned)

f. Mapping strategy to the Rosotravel model

Below is a breakdown of how to map individual types of Viator destinations to destinations on the Rosotravel website:

- Country: Country, Island
- Region: Region, AREA, COUNTY, DISTRICT, PENINSULA, PROVINCE, STATE, UNION TERRITORY
- City: City, HAMLET, NEIGHBOURHOOD, NATIONAL PARK, TOWN, VILLAGE, WARD

g. Data model

01. Destination (Rosotravel)

destination_id (PK)

type: COUNTRY | REGION | CITY

name

slug

parent_destination_id (nullable)

status: active | inactive

02. DestinationMapping (Viator)

destination_id (FK → Rosotravel)

viator_destination_id

viator_destination_type

Rules:

- One Rosotravel destination can have multiple Viator mappings.

h. System behaviour when changes are made on the Viator side

01. Deletion of a destination by Viator

If Viator deletes a destination (e.g. a country):

- Viator affiliate products assigned to that destination are deactivated,
- the Rosotravel destination is not deleted,
- Rosotravel products remain active.

02. Changes to the structure or types of destinations

If Viator:

- changes the destination type,
- changes hierarchical relationships,

then:

- only the entries in DestinationMapping are updated,
- the Rosotravel destination_id remains unchanged,

i. To be confirmed with the developer during the project

To be verified during integration:

- How many unique REGIONS does Viator return globally?
- Does Viator provide a hierarchy of regions (region → subregion)?
- Does DISTRICT always have a parent CITY, or sometimes a REGION?
- Is ISLAND ever the root, or does it always have a COUNTRY?
- Does NATIONAL PARK always have a parent CITY or REGION?
- We put WARD destinations in City, but we don't really want that if they assign trips to them. Check in the sandbox what these wards are in the context of destinations.

11. Changes regarding automatic location search in the search engine and product surveys

a. Data Layer vs Frontend API

01. Core Data Platform (integration backend – Viator Layer)

The developer's scope includes:

- Integration with Viator API (products, destinations, availability, pricing, recommendations).
- Deterministic data retrieval, versioning and storage (RAW / OFFER / REALTIME).
- Maintenance of an internal location database (countries, cities, regions, POI, meeting points, pickup points).
- Deduplication and maintenance of relationships between entities.
- Synchronisation and updating of data in accordance with Viator Partner API guidelines.
- Ensuring data integrity and consistency as a Single Source of Truth.

b. General principles and purpose

01. General principles

- **Automatically retrieve locations from the Viator database** and store them locally. This also applies to destinations that Viator has from the Google database.
- **Enabling the tour search engine to use these locations** (autocomplete + filtering by country/city).
- **Enabling checkout/pickup/meeting point forms** to search in the same integrated location database.
- **Downloading and storing recommended products from Viator** and the ability to connect them to our search engine/listings.
- **Elements retrieved via the API should be updated in accordance with the [Viator Partner API](#) section.** It is crucial to adhere to the time frames specified there for individual endpoints.

02. Objective

The objective is to extend the functionality of the tour search engine and forms (e.g. selection of meeting point, pick-up or drop-off point) with automatic location data and recommended products from the Viator database.

The system should independently retrieve and update data on locations, places, countries and other regions, as well as recommended products, and then use them to search for and present tours on the website. Please check their database of regions here, and then we will decide whether to add anything, but within the database itself, we want to add every destination, e.g. Tenerife, if it counts as an island.

c. Automatic location collection

01. General description

The system is to have **an internal database of locations** that are automatically downloaded and updated from the Viator database. Locations are to include meeting points, pick-up points, sightseeing locations, as well as general cities or geographical points used in product descriptions.

02. Scope

- Automatic data collection
 - The system automatically downloads location data from the Viator database at regular intervals.
 - For each new or updated product, the system recognises all locations (e.g. start point, end point, pick-up point, meeting point).
- Creating and maintaining an internal location database
 - All collected locations are to be stored in a special database/table in the Rosotravel system. This should apply to both locations retrieved by the Viator API and (if possible) the Google API
 - Each location should be saved with complete information (name, country, city, street, coordinates, data source, location type).
 - In the event of data updates, the location should be automatically refreshed – without duplication.
 - The system should recognise and link locations to the appropriate cities and countries (e.g. the location "Piazza Venezia" assigned to the city "Rome", country "Italy").
- Use of locations in the system
 - Location data is to be used by the trip search engine – when the user enters the name of a place, city or country, they should receive suggestions from this internal database.
 - The same location data is to be available in trip forms – e.g. when selecting a pickup location, meeting point or drop-off location. Here it is specified what the data will be used for.
 - The system is to recognise which locations are associated with particular products and allow trips to be searched for based on the selected location.
- Data update and maintenance
 - The update process is to run automatically and in the background so that the location database is always up to date without the need for manual intervention.
 - If new locations appear in Viator's data, the system should add them automatically.

- For locations that no longer exist or have been marked as inactive, the system should mark them as outdated but not remove them from the history.

03. Expected result

- The system contains a **complete, up-to-date location database** including cities, meeting points, pick-up points and other locations from Viator.
- The search engine and forms (e.g. 'meeting point', 'pickup') use this data so that the user can see current location suggestions.
- The system keeps the data up to date without the need for manual file uploads.

d. Recommended products

This is all excluded, I am leaving it for the archive in the comment

12. Flow for bookings between Rosotravel and Viator

General assumption of the system:

The Rosotravel system is a "front-end" system: every operation performed by a customer on a product originating from Viator (creation, change, cancellation, clarification of details) is first recorded by us and then reflected in Viator; statuses and final decisions (availability, possibility of change, refund amount) come from Viator and must be periodically downloaded to our system so that the customer always sees the current status.

The term "system" means that Rosotravel acts as an intermediary layer between the customer and the Viator API:

- all operations initiated by the customer are first recorded in the Rosotravel system,
- the Rosotravel system forwards requests to Viator,
- Viator remains the source of final decisions for booking availability, booking confirmations, booking changes, cancellations, and refund amounts. This is to prevent situations where, due to a system error on our website, a tour appears to be available in Rosotravel, while it is no longer available in Viator.
- Booking statuses are synchronised with Viator at a set frequency (polling) rather than in guaranteed real-time mode.

As a merchant, Rosotravel is responsible for sending all confirmations, cancellations, and changes made directly by the supplier or Viator to the customer. Viator only provides us with updates via API (and possibly email), but the final message to the customer is sent by our Rosotravel system.

a. Customer booking

01. Purpose of the flow system

The customer makes a booking on rosotravel.com, we sell Viator's product, so we need to have confirmation from Viator (sometimes immediately, sometimes after a while), and then be able to show it to the customer and have it in the accounting and invoicing system.

Any change in the status of a booking on Viator's side must automatically trigger the sending of an appropriate message to the customer from our system.

Our system should be able to handle all types of bookings:

- **Instant confirmation:**
 - Instant confirmation products are confirmed automatically at the time of booking, and the customer should be charged immediately.
- **Manual confirmation**
 - Manual confirmation products are those that only operate at the discretion of the supplier, who must confirm or reject each booking request manually. Once the booking is requested, it will remain in a 'pending' state until actioned by the supplier; or, if no action is taken by the supplier, the time-out period (72 hours) is exceeded. The customer should only be charged once confirmation is received.
- **Instant-then-manual confirmation**
 - Instant-then-manual confirmation products behave like instant confirmation products up until a certain time prior to the product's starting time. From that point on, the product will require manual confirmation by the supplier.

02. Flow

➤ The customer selects a product on the Rosotravel website

- The product is in our database, but its "source of truth" is Viator (price, availability, rules).
- The customer selects: date, number of people, product option, language, and possibly pickup.

➤ Our back-end checks availability in Viator in real time

- We send Viator: which product, when, and with which group of travellers.
- **Expected result:** Viator sends back current availability + current price + currency + possibly information on whether it is an instant booking or "pending confirmation".

➤ If there is availability, our system creates a "pending booking" on our side.

- We create a booking record with a working status of "pending / awaiting Viator".
- We save what the customer has entered (traveller details, pickup, answers to questions, contact details).
- **Our system sends the customer's details and trip details to Viator.**
 - We transfer: product, option, date, time, pax mix, answers to booking questions (e.g. names, weight, pickup, arrival mode), customer contact details (email/phone), our partner booking number (for later identification).
 - **Expected result:** Viator accepts the booking request and sends us their booking number and status.
- **Viator communicates the status**
 - **Option A – immediately:** Viator immediately sends back the status "confirmed" → our booking changes from "pending" to "confirmed", the Rosotravel system sends a confirmation to the customer (not Viator) at this point, and we can immediately collect the payment or complete it if it was pending.
 - **Option B – upon confirmation from the supplier (Manual confirmation and Instant-then-manual confirmation):** Viator sends back the status "pending" → our booking remains in the status "awaiting confirmation from Viator". From this point **on, we have to periodically ask Viator about the current status**. The customer receives the information "booking accepted, waiting for confirmation from the organiser" from us.
- **Viator communicates with the supplier**
 - This is an internal part of Viator – we do not interfere here.
 - The supplier can confirm, reject or request additional information.
 - The result is returned to Viator.
- **Our system periodically checks with Viator to see if the status has changed**
 - If the status changes to "confirmed" → we update the booking + send confirmation to the customer.
 - If the status changes to "rejected" → we update it to "rejected by supplier (Viator)" + message to the customer from Rosotravel + possible refund/release of payment.
- **Communication with the customer**
 - Any change in status on the Viator side must be reflected in Rosotravel and sent to the customer (e-mail – depending on what we agree).

(Additional issues such as panel/SMS to be determined in the future if necessary)

- In particular: "accepted", "confirmed", "rejected". Viator does not send these notifications to our customer on our behalf – this is done by the Rosotravel system.

b. Cancellation of a trip by the customer

01. Purpose of the flow system

The customer wants to cancel, but the product is Viator's, so the refund and terms and conditions come from Viator. We only act as an intermediary and show the result, but we are responsible for communicating the information to the customer.

02. Flow

- **The customer clicks "cancel" in Rosotravel**
 - They select a reason (we provide a list of reasons).
- **Our back-end first retrieves information from Viator about whether the booking is currently cancelled and under what conditions**
 - **Expected result:** Viator sends back: "can be cancelled" / "cannot be cancelled" + refund amount + currency + possibly 0% if it is too late.
- **We show this to the customer (or do it immediately if we set "one-click")**
 - If the customer accepts, we proceed.
- **Our system sends a cancellation request to Viator**
 - We include the reason for cancellation (Viator requires this because it passes it on to its processes).
- **Viator cancels on its end and updates the booking status**
 - If cancellation is permitted → status "cancelled/accepted".
 - If not → information that cancellation is not possible.
- **Our system retrieves the result from Viator and saves it in our database**
 - The booking receives the status "cancelled by the customer (by Viator)".
 - We save the refund amount to pass it on to accounting/payment gateway.
- **We initiate a refund to the customer in accordance with the information from Viator**
 - If Viator refunded 100%, we refund 100%.
 - If 0% – we communicate 0%.
 - If partial – we refund partially.
- **The customer receives confirmation from Rosotravel**
 - With the Viator booking number (so that support can communicate) and information about the refund.

Business requirement: the cancellation message and refund amount are sent by the Rosotravel system, based on the data returned by Viator.

c. Scope: Viator Supplier Communications (CLC) + CC to Rosotravel

1) Objective

- Enable **Closed-Loop Communication (CLC)** for Viator bookings.
- The supplier communicates **directly with the customer**.
- Each message from the supplier is to be **CC'd to Rosotravel** at: **tours@rosotravel.com**.

2) Scope

- Applies only to bookings made through Viator integration on endpoints:
 - POST /booking/book
 - POST /bookings/cart/book

3) Integration requirement (request body)

For each Viator booking, the integration **MUST** send a **communication** object:

- communication.email
 - format: "{customerEmail},tours@rosotravel.com"
 - no spaces
 - order: **customer**, then **CC**
 - **tours@rosotravel.com** must be **only CC** (never as the only address)
- communication.phone
 - **customer's telephone number** in **E.164** format (e.g. **+48500345300**)
 - should come from the data provided by the customer at checkout/booking

4) Configuration (ENV/Config)

- VIATOR_CLC_CC_EMAIL="tours@rosotravel.com"

Do not hardcode in the code — value from the configuration.

5) Validations (before sending to Viator)

- customerEmail
 - **MUST** exist and be a valid email address
- customerPhone
 - **MUST** exist and be in E.164 format (+ + digits, no spaces)
- VIATOR_CLC_CC_EMAIL
 - **MUST** exist and be a valid email address

If validation fails:

- interrupt the booking flow (validation error on the integration side)

6) Payload example (fragment)

```
{  
  "communication": {  
    "email": "customer@example.com,tours@rosotravel.com",  
    "phone": "+48500345300"  
  }  
}
```

7) Acceptance criteria

- **communication** is sent in every request to **POST /booking/book** and **POST /bookings/cart/book**.
- **communication.email** always has the format **customerEmail,tours@rosotravel.com** (no spaces, correct order).
- **communication.phone** always contains **the customer's telephone number** in E.164 format.
- **tours@rosotravel.com** never replaces the customer's email (it is only CC).
- The absence of **customerEmail** or **customerPhone** blocks the booking (we do not send a request to Viator).

13. View all trips (Manage Products)

Products

[+ Create new product](#)

Filter by product

Filter by status

All products (26)

Bookable products (25)

Updates needed (8)

Not yet submitted (1)

Expired (0)

Expiring soon (0)

Not bookable (0)

Sort Title ▲ ▼

1 - 10 out of 26



DKCO010 | Copenhagen Day Trip to Frederiksborg Castle by Private Car

☆☆☆☆☆ Not rated

Bookable



DKCO002 | Grand Bike Tour of Copenhagen Old Town, Attractions, Nature

☆☆☆☆☆ Not rated

Bookable



DKCO013 | Copenhagen Day Trip to Malmo Old Town & Castle by Train/Car

☆☆☆☆☆ Not rated

Bookable



DKCO016 | Best of Odense Day Trip from Copenhagen by Car or Train

☆☆☆☆☆ Not rated

Bookable



T-560098 | test

☆☆☆☆☆ Not rated

Not yet submitted

Incomplete Product: This product creation is incomplete, please click 'Resume Create Wizard' to finalize your product creation.



DKCO003 | Family Tour of Copenhagen Old Town, Nyhavn with Boat Cruise

☆☆☆☆☆ Not rated

Bookable



DKCO001 | Guided Car Tour of Copenhagen City Center, Nyhavn, Palaces

☆☆☆☆☆ Not rated

Bookable



DKCO005 | Copenhagen Old Town, Nyhavn, Canal Walking Tour & Christiansia

★★★★★

Bookable



DKCO012 | Danish Food Tasting and Copenhagen's Old Town, Nyhavn Tour

☆☆☆☆☆ Not rated

Bookable



DKCO008 | WWII Copenhagen Nyhavn and War Museum Private Walking Tour

☆☆☆☆☆ Not rated

Bookable

<< < 1 2 3 > >>

a. We want to add an additional filter to the view of all trips

01. Filter by origin

- The filter is a dropdown menu with options to choose from
 - Rosotravel
 - Viator

02. Information about displaying the page

- next to the stars, there is an average rating, e.g. 4.6/5
- The page displays 50 items, not 1-10, but 1-50

- Customer ratings, text and quantity must be displayed exactly as they appear on the partner's website. This element must not undergo any changes whatsoever, such as automatic translation or changes to the name, image, etc.

14. Adding a trip (Create Product) - General rules

a. Selecting the trip type (product category)

Below is what the selection scheme should look like, followed by the changes to it.

The screenshot shows a web form titled "Create a new product". Below the title is a question: "Which best describes your activity?" followed by a subtext: "This helps us categorize your product so customers can find it." The form contains a list of radio button options, each with a title and a descriptive subtext:

- ☐ Attraction ticket
Like entry to a landmark, theme park, show
- ☒ Tour
Like a guided walking tour, day trip, city cruise
- ☐ City card
A pass for multiple attractions or transport within a city
- ☐ Hop-on hop-off ticket
Entry to a hop-on hop-off bus or boat
- ☐ Transfer
Transportation services like airport or bus transfers
- ☐ Rental
Like modes of transport, sports equipment, Wi-Fi routers
- ☐ Other
Like a cooking class or multiple activities sold together

A blue "Continue" button is located at the bottom right of the form.

01. Change in nomenclature:

- Which best describes your activity -> Identify the source of activity
- This helps us categorise your product so customers can find it. -> Please specify if you would like to manually add a Rosotravel product or if you would like to add a partner tour via the link API.

02. Changes to the list layout

- Attraction ticket, Tour, City Card, Hop-on hop off ticket, Transfer, Rental, Other should be changed to two options:
 - Rosotravel
 - Affiliate/Partner Product

03. How the list works.

- The Rosotravel and Affiliate/Partner Product options should be manually selected by the user, and both serve as dropdown menus with the following options
 - Rosotravel
 - Tour
 - Affiliate/Partner Product
 - Viator

The user can only select one of the main options Rosotravel or Affiliate/Partner Product

- Rosotravel option.

In the case of the **Rosotravel** option, it should be coded in such a way that the next categories, such as Ticket, City Card, etc., will appear in subsequent stages. Currently, every Rosotravel tour that has already been created should automatically have the Tour type, and new products will have the option to choose. Selecting, for example, an attraction ticket will have different fields added to the survey than the product.

So, currently, when we click on the Rosotravel option, a list should expand where there is only Tour, which should be automatically selected. The user can click Continue without any additional selection.

- Affiliate/Partner Product option.

In the case of the **Affiliate/Partner Product** option, it should be coded in such a way that when we click on it, a list with the options Musement, Tiqets, Klook, GetYourGuide, Viator, Others expands. The user should select one of the options and then click Continue.

We are currently planning to program the API for Viator and then for Tiqets. In this case, I present how the surveys for each option should be coded at the very beginning:

- Viator/TripAdvisor - API programming rules in accordance with the [Viator API link](#)
- Tiqets - Inactive
- Klook - Inactive
- GetYourGuide - Inactive
- Musement - Inactive
- Others - Inactive

This should be prepared in such a way that other surveys for individual options can be programmed in the future.

15. Adding a Viator/TripAdvisor affiliate tour (Create Product)

a. General rules for adding affiliate tours

- Viator affiliate products are added in whole bundles divided by destination (country).
 - The result of downloading such a bundle should be a list of tours that the bundle applies to

- Each tour from the bundle should be added automatically, unless otherwise specified
 - Each element of the tour should be downloaded 1:1 as it is in the Viator API.
 - Trips added from the package should have the status "inactive" and should be added as a draft ready to be launched
 - All affiliate products have the same sub-items as manually added products, but are filled in automatically. The following point-by-point breakdown shows the changes from the standard version
 - Affiliate tours should be updated on an ongoing basis to be consistent with the partner's website. This should be consistent with the API from the relevant provider
- The entire package and all its elements should be pulled automatically in accordance with the API. In this case, the elements that must be pulled via the API are:
 - Tour title
 - Full trip description
 - Availability
 - Price list
 - Meeting point
 - Attractions
 - Number of participants
 - Languages
 - Review
 - product rating
 - product photos
 - tags
 - product destination from the Viator database (including those that Viator has from Google)

b. Tour type selection (product category)

This stage was defined [in the previous section 6a\).](#)

c. Tour title and product code (basic information title)

01. General rules:

- The tour title should be retrieved by the API 1:1
- The product code should be retrieved via API and remains 1:1 as it is in the Viator API

d. Description and highlights (basic information description & highlights) - Affiliations

Affiliation system operation:

- Highlights - The entire text should be retrieved by the API code
- Short description - The entire text should be retrieved by the API code

- Description - The entire text should be retrieved by the API code. Additionally, in the case of Viator, we have the following division:

e. Itinerary - Viator Affiliations

The Itinerary view is exclusive to the survey when we add a Viator product and should not appear in the future when selecting other partners

01. Itinerary view diagram

We want to model the basic scheme on Viator, as in the example below

×

Edit your itinerary

Be sure to enter all key attractions in your itinerary. This helps travelers find and book your experience.

What is the total duration of your tour from start to end, including pickup and drop-off (if applicable)?

☒ Flexible

to

What attractions will travelers see?

Enter all landmarks or points of interest in your itinerary.

- The Rembrandt Hotel, 11 Thurloe Pl, South Kensington, London SW7 2RS, UK**
✎ 🗑

Meet your guide in front of The Rembrandt Hotel, 11 Thurloe Pl, South Kensington, L...
10 minutes, Admission is free
- V&A - Victoria and Albert Museum, Cromwell Road, London SW7 2RL England**
✎ 🗑

Private tour of the Victoria and Albert Museum to immerse yourself in art history ...
80 minutes, Admission is free
- South Kensington, London England**
✎ 🗑

Enhance your V&A experience with the highlights of South Kensington. The Victori...
1 hour , Admission is free
- London, London, England**
✎ 🗑

Book a 1,5-hour round-trip transfer and a 2-hour private tour of the rich collection...
1 hour , Admission is free
- Royal Albert Hall, Kensington Gore, London SW7 2AP England**
✎ 🗑

We also offer a group tour of the Victoria and Albert Museum and South Kensingt...
1 hour , Admission is free

+ Add an attraction

To be changed:

- The entire field "What is the total duration of your tour from start to end, including pickup and drop-off (if applicable)?" - **to be removed**
- The rest remains unchanged

View operation:

- As shown in the diagram, it should be possible to add attractions using the "+ add an attraction" button, and then, after clicking it, we get the view shown below.
- Each attraction should have a button next to it that allows you to edit or remove it from the list. Activating the delete button should require additional confirmation



Add an attraction

A detailed itinerary helps travelers know what to expect. [Learn more](#)

Select an attraction

Describe what travelers will see and do here if they book your experience

Enter a description about this attraction only. For example, "Our accredited guide will meet you at the ticket booth and will accompany you up the tower, explaining what you'll see at each level."



How much time do travelers typically spend here?

☐ Pass by without stopping

minutes 

Is admission to this place included in the price of your tour?

- ☐ Yes
- ☐ No
- ☐ N/A (Admission is free)

Cancel

Done

- We want to model the above attraction field on Viator. The default goal is for it to be filled in automatically when downloading products via API

f. Location (city, country) and attractions (location) - Affiliations

Affiliation system operation

01.City and country categories:

- The country or region and city on our website should automatically be assigned to a trip based on the location from the API and mapping.

02.Attraction categories - Viator:

- In the Raw scheme, you retrieve Viator categories in a 1:1 format without any changes

g. Tags, sections (keywords) - Affiliates

- At this stage, the system only maps tags 1:1 from the Viator API and transfers the package to the tour/tag category view. The whole process is described in section [Category mapping](#)

h. Included in the price and not included in the price (Inclusions) - Affiliations

- The system should automatically retrieve everything after the API code, without any changes.

i. Cancellation Policy - Affiliates

Affiliate system operation:

- The system should retrieve everything automatically, without any changes. As it is on the affiliate partner's website. This cannot include our cancellation policy, as the cancellation policy on the partner's website may be different. All cancellation methods will be discussed in this section.
- The cancellation system flow is described in section [II.4.c.02](#)

j. Important information (additional information) - Affiliates

Affiliate system operation:

- The system should retrieve everything via API without any changes.

k. Photos - Affiliates

Affiliate system operation:

- Each package should be imported without any changes. **Important:** Photos should be as they are via the API.

l. Options - Affiliates

How the affiliate system works:

- The system should automatically retrieve everything after the API code, without any changes.

m. Meeting & collection point - Affiliates

Affiliate system operation:

- The system should automatically retrieve everything after the API code, without any changes.

n. Pricing - Affiliates

01. General rule for price retrieval

For all Viator affiliate products, the Rosotravel system uses:

price.original.recommendedRetailPrice as the default price displayed on:

- listings (home page, country, city, attraction, shop),
- the product page (before the customer selects a date),
- anywhere where the "price from" is displayed.
- By default, we retrieve the value for the Adult category, and if it is not available, we use the next available one in the order: ADULT → SENIOR → YOUTH → CHILD → INFANT.

The purpose of this action:

recommendedRetailPrice is the public price suggested by Viator — the safest starting price before checking availability.

02. Real-time price update (required by Viator)

Before finalising the payment, the system must:

Perform an availability check for:

- the selected date,
- the selected option,
- number of travellers and age categories

03. Cache:

- The listing price should be stored in the cache for up to 24 hours.
- The price on the product page must be checked every time the page is changed or refreshed
- The price in the basket must be checked each time "Add to basket" is selected.
- The price at checkout must be checked again at the final check stage.

04. Currency conversion

- Currencies should be converted in the same way as the Viator system converts them using the API code. The conversion should take place **after** the correct price has been selected.

r. Availability - Affiliates

01. Expected result:

- For affiliate products, Rosotravel does not maintain or use its own availability.
- Availability is entirely controlled by the partner (e.g. Viator) and checked exclusively by their API.

02. Back-end operating principles

- Product type identification:
Each product must have a clearly defined type:
 - ROSOTRAVEL – own product
 - AFFILIATE – partner product (e.g. Viator)
- Availability for affiliate products:
For AFFILIATE products, Rosotravel's internal availability system is inactive.
 - no own calendars,
 - no hourly slots,
 - no seat limits on our side,
 - no manual editing of availability in the admin panel.
- Administration panel:
All elements related to availability for affiliate products must be:
 - hidden, or
 - locked (read-only).
- Source (single source of truth):
The only source of information about availability is the partner's API (Viator), in particular:
 - available dates and times,
 - number of available places (if returned by the partner),
 - price returned together with availability,
 - booking confirmation logic (instant/manual/mixed).

03. Front-end operating principles

- Real-time verification rule:
Any change made by the user that may affect availability must trigger a re-check of availability by the partner's API, including:
 - change of date or time,
 - change in the number of participants,
 - change in age categories (including Senior),
 - change in product options or language,
 - changes to pickup/meeting/end point, if they affect the availability of a given trip.

- Cache (optional):
 - only short-term cache is permitted in order to limit the number of queries,
 - cache must never overwrite partner data,
 - in case of discrepancies, the response from the partner's API always takes precedence.

04. Error handling

- If the partner's API:
 - does not respond,
 - returns an error,
 - does not confirm availability,
- then the availability of the affiliate product should be treated as unknown/unconfirmed, and the booking process cannot be continued until a correct response is received from the API.

s. Changes for Facility Duration

01. Duration

- General rules:
 - **Duration:** The field is automatically populated based on the duration from the API code and ranges from the shortest duration of the option to the longest duration of any of the options.

t. Administrator's opinions - new view - Affiliations

Affiliation system operation:

- The system should retrieve all reviews that have been given to the product on the day the trip is added via the API. Reviews are transferred 1:1 without changes and should be marked accordingly that the source is Viator

u. Model for updating products and supporting data for Viator products - Affiliates

01. Model assumptions

Build a backend mechanism for synchronising data with Viator so that:

- the product catalogue, availability and booking information are up to date,
- integration is resistant to rate limiting and overloads,
- the system detects new references (tags, locations, attractions) from the Viator API
- queries are not "too frequent" (in accordance with Viator guidelines).

02. Overall solution architecture

The developer is to prepare 4 elements:

- Update schedule

Separate schedules for:

- product deltas,
- availability deltas,
- booking change deltas,
- full refreshes of dictionary data within the Viator API.

- Job queue

Each update works as a job, with:

- retry,
- concurrency control,
- priorities,
- rate limiting protection.

- Checkpointing / state store

Separate checkpoints for the "last processed change" for each data type:

- products,
- availability schedules,
- booking modifications,
- dictionary data (last full refresh).

- Reference resolver (on-demand)

- A mechanism that detects new references in the data and orders immediate dictionary updates (e.g. new tags, new location codes, new destinations/attractions).

03. Start of integration — Initial ingestion (first full feed)

- Expected result. The system performs the first full feed:
 - products,
 - related data necessary for the catalogue to function (e.g. tags, attractions/destinations, availability/schedule),
 - auxiliary data needed to build listings and a search engine.
- Rules
 - Initial ingestion must end with setting start checkpoints for delta mode.
 - After initial ingestion, the system switches to delta updates + periodic full refresh of dictionaries mode.
- The scope of initial ingestion includes only:
 - Full download and saving of snapshots:
 - products (RAW + OFFER layer),
 - related OTA entities (destinations, attractions, tags, suppliers – within the scope provided by the Viator API),
 - required relationships (parent relations, relationship edges).
 - Setting start checkpoints for delta mode:
 - products,
 - availability,
 - booking modifications,

- dictionaries.
- **Initial ingestion limitations**
 - Availability/schedule
 - Initial ingestion does not include downloading complete, multi-month availability calendars.
 - Availability is initialised to the minimum operational extent required.
 - Full availability updates are performed in delta/realtime mode according to the cadence.
 - Initial ingestion **does not mean downloading** the **entire** Viator **global** catalogue.
 - Initial ingestion is triggered for defined "ingest packages" (e.g. destinationId / country / city / list of destinations) and only includes products available within that scope.
 - The scope of the package is configurable and constitutes input for the ingestion process.
- **Data for listings and search engines**
 - Initial ingestion provides source data (RAW/OFFER/RELATION).
 - The construction of search indexes, presentation layers and frontend optimisation is the responsibility of the API/application layer.
- **Product scope**
 - Initial ingestion covers products available through the Viator API within a defined scope (e.g. assigned destinations).
 - It does not include external enrichment or data from outside the Viator API.
- **No enrichment**
 - Initial ingestion does not include integration with Google Places, Wikidata, or other external sources.
 - The reference resolver only works within the scope of references provided by the Viator API.
- **No business workflow**
 - Initial ingestion does not include moderation, ranking, recommendation, or CMS processes.
 - The system stores data deterministically in RAW/OFFER/REALTIME layers.

04. Cyclical updates — Delta updates (polling)

- **Expected result**
 - The system periodically retrieves and processes only changed records, in accordance with Viator's recommended cadence.
- **Cadences (target, configurable)**
 - Product changes: every 15–30 minutes
 - Availability schedule changes: every 15–30 minutes

- Booking changes: every 5 minutes
- Acknowledge booking changes: automatically after successful processing of the change package (cycle closure)
- Rules
 - The system should not query more frequently than recommended (integration protection).
 - Updates must be idempotent (reprocessing the same change does not corrupt the data).

05.Periodic full refresh of dictionary data (detection of changes and deletions)

- Expected result

The system periodically refreshes auxiliary/dictionary data with a full download in order to:

- detect changes not covered by deltas,
- detect deletions,
- maintain consistent dictionaries for mapping.
- Frequencies (target, configurable)
 - Booking status: hourly
 - Exchange rates: daily + on-demand if the rate has expired
 - Destinations / Attractions / Recommendations / Tags / Reviews / Suppliers search: weekly
 - Booking questions / Locations bulk / Cancel reasons: monthly

06.On-demand updates (when a new reference appears)

- Expected result

If a reference appears during ingest/delta that is not in the Rosotravel database, the system:

- mark it as a "missing reference",
- adds the task to the on-demand queue,
- after completing the dictionary, it returns to processing the main record (or completes it after the fact).
- Examples
 - new location code → pull location details (preferably in batches),
 - new tag → pull tags and submit for mapping,
 - need for calculations, and the exchange rate has expired → pull exchange rates immediately.

07.Rate limiting and integration resilience

- HTTP 429 (Too Many Requests)

Expected behaviour:

- the system reads the limit headers and applies backoff exactly according to Retry-After,

- does not perform aggressive retries,
- maintains separate limits per job type (per "update category") so that problems with one type do not block the entire synchronisation.
- HTTP 503 (Service Unavailable)
 - Expected behaviour:
 - the system applies Retry-After,
 - the job goes to the retry queue with increasing delay,
 - does not cause cascading overload (concurrency control).

08. Processing logic — data quality rules

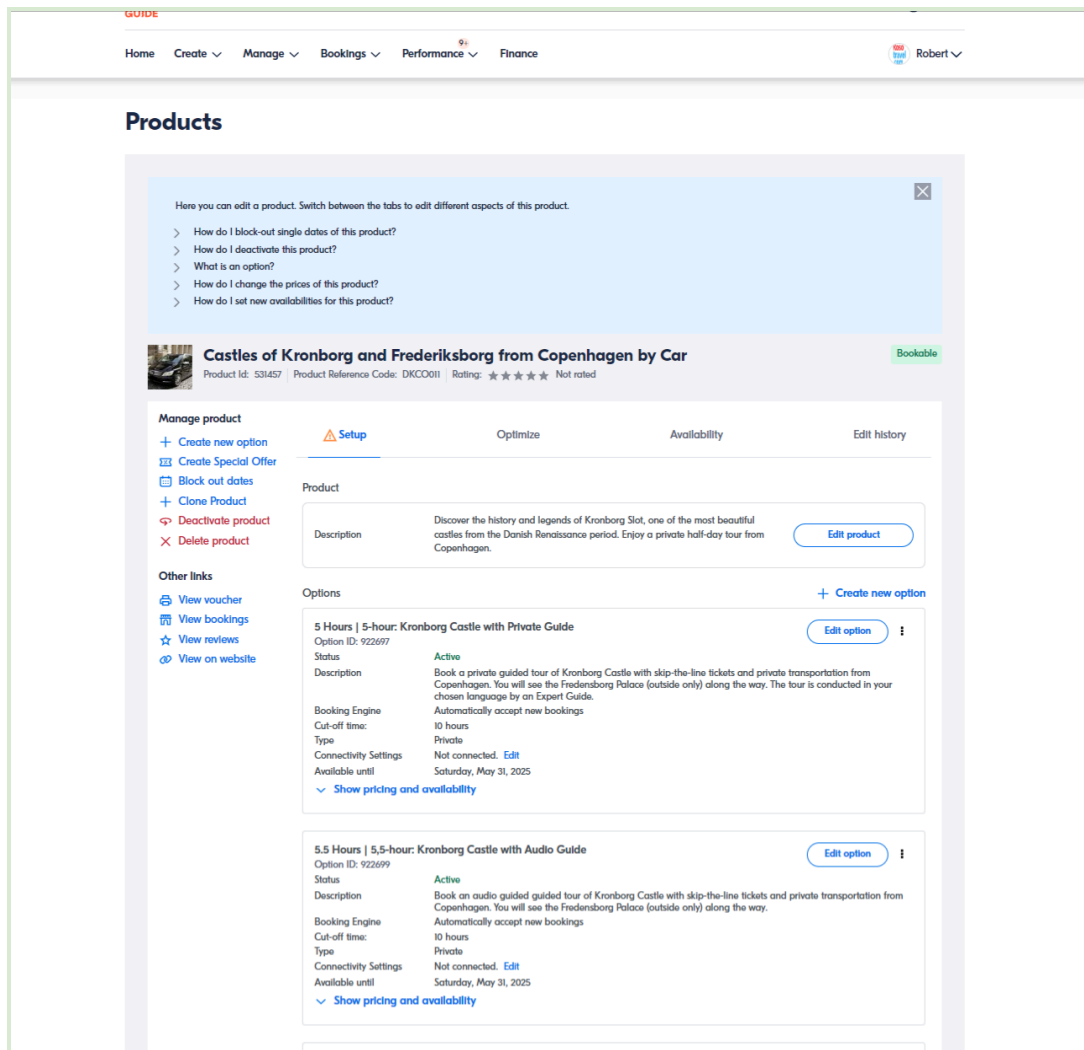
- Processing order: first the product base data, then the dependent data (availability, reviews, recommendations), and the resolver fills in the missing references.
- Autonomy at the record level: product/availability records should be consistent (not partial).
- Versioning/auditing (recommended): store the raw feed and update metadata (timestamp, source, update type) to be able to debug differences.
- The developer's responsibility is limited to:
 - technical logs
 - integration metrics
 - job statuses
 - retries / rate limits

v. Definition of "done" (acceptance)

We consider the functionality to be implemented when:

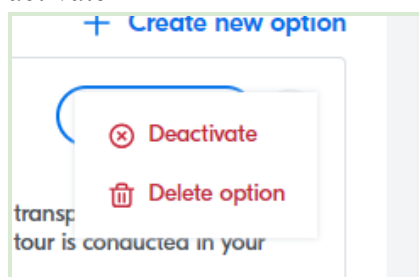
- initial ingestion works end-to-end and sets checkpoints,
- delta updates work cyclically and do not duplicate data,
- full dictionary refresh works according to schedule,
- on-demand resolver completes missing references,
- integration correctly handles 429 and 503 (Retry-After),
- Monitoring (logs/metrics) for stability control is visible.

16. Single trip management panel - Affiliations



In the case of an affiliate trip, the management panel looks almost identical with the following changes. Initially, the affiliate product added as a draft is inactive:

- The three dots give you the following options:
 - deactivate
 - activate



17. Changes in the view of all bookings

a. Additional filtering functions

- filter by sales channel

- Rosotravel
- Viator

18. Changes to the view of all ratings by affiliation

The screenshot shows a web interface for viewing travel reviews. On the left, there are several filter sections: 'Filter by product' with a search bar, 'Filter by option' with a search bar, 'Filter by travel date' with 'From...' and 'To...' date pickers, 'Filter by review date' with 'From...' and 'To...' date pickers, and a 'Rating' section with buttons for 5 stars, 4 stars, 3 stars, 2 stars, and 1 star. The main content area on the right has a header with 'OVERVIEW', '4.5/5 average rating', and '1443 reviews written'. Below this is a 'Sort' dropdown set to 'Most recent first' and a pagination indicator '1 - 10 out of 1443'. The review list shows three entries, each with a star rating, date, and details. The third review is expanded, showing a detailed text review, a 'Value for money?' rating of 3.0/5, and booking details: 'Option: English | Schonbrunn Palace and Gardens Tour in English', 'Booking: GYBLKXYFGA', 'Travel date: Friday, January 26, 2024 - 11:05', and 'Traveler type: Family with older kids'.

a. Additional filtering functions

- filter by sales channel:
 - Rosotravel
 - Viator

b. Change to the rejected button

The rejected button is disabled for Viator reviews; it is not possible to exclude them on the website, even using the admin panel.

19. Flow for transactions detected as fraudulent (This is Viator's internal system, not ours)

If the Viator system detects a transaction suspected of fraud:

- Viator will automatically cancel the booking on its side (cancellation status returned by the Viator API).
- Information about cancellation due to fraud:
 - is available in the Viator API response / when re-downloading the booking status,
 - it should be handled by the Rosotravel backend as an external system cancellation.
- Upon detection of such a cancellation, the Rosotravel backend:
 - updates the booking status in the database to cancelled (fraud – Viator),
 - blocks further operations on the booking (changes, reconfirmations),
 - sends an automatic email notification to the support team at bookings@rosotravel.com with information about the cancellation caused by Viator detecting fraud.

20. Booking Change System on the Website (Rosotravel) (Back-end and Front-end section)

a. Context and Goal

01. General Objective

In the new Rosotravel version, customers can book products directly on rosotravel.com.

The booking change system must allow the customer to:

- modify selected booking parameters,
- view bookings in their account and on a dedicated booking details page,
- receive statuses, vouchers, and notifications after changes,
- handle additional charges and refunds,
- store each successful change as a new booking version.

02. Product Type Distinction

Rosotravel sells two product types, therefore the system must distinguish between:

- **Rosotravel** — own products, where Rosotravel rules decide,
- **Viator** — affiliate partner products, where Viator and supplier rules decide.

03. Shared and Separate Logic

Because of this distinction, there are two separate booking change flows.

At the same time, both flows must share common rules in relation to:

- user experience,
- versioning,
- pricing difference calculation.

b. Scope of Customer Changes

01. Parameters the Customer Can Change

The customer must be able to change:

- number of participants,
- tour date,
- tour time,
- tour language,
- pick-up, if applicable,
- drop-off, if applicable,

- participant name and traveler data, to the extent required for service delivery.

02. Parameter the Customer Cannot Change

The customer must not be allowed to change the tour option.

03. Rule for Option Changes

If the customer wants to change the tour option, the system must require the following path:

- cancel the booking,
- create a new booking with a different option.

c. Change Entry Points and UX Requirements

01. General UX Requirement

The developer must design the booking change interfaces.

By default, the customer must be able to request booking changes in at least two ways.

02. Dedicated Booking Page

The system must provide a dedicated booking page containing:

- booking details,
- action buttons such as “**Change booking**” and “**Cancel**”.

This page must be accessible:

- from the booking list,
- from links included in emails.

03. Customer Account – My Bookings

The system must also include a dedicated **My Bookings** area within the customer account.

This area must contain:

- a bookings list page,
- the ability to open booking details from the list,
- the ability to request changes or cancellations from the booking details view.

04. UX Consistency with Cancellation Flow

The cancellation system already exists.

The booking change system must therefore remain consistent with the cancellation UX structure, specifically:

- list,
- details,
- actions.

d. Common System Rules Applicable to Both Rosotravel and Viator Products

01. Master Validation Rule

Regardless of product type, every booking change attempt must always be checked before finalization.

The validation must confirm:

- whether the requested change is possible for the booking,
- whether the booking status allows changes,
- whether time limits allow the change,
- whether the change complies with the relevant change policy,
- whether any product-specific constraints apply.

Only after a positive validation result may the customer proceed to finalize the change, including any surcharge or refund handling.

02. Booking Versioning

Each successful change must be stored as a new booking version in the Rosotravel database.

02.1. Scope of Stored Data

Each new booking version must include the final parameters, including:

- date,
- time,
- language,
- participant count,
- pick-up,
- drop-off,
- traveler data,
- final price.

02.2. History and Audit Trail

The system must preserve change history for:

- customer support,
- accounting,
- internal operational review.

02.3. Voucher Regeneration

A new voucher must be generated after every successful booking change.

03. Financial Rules After a Change

The system must support the following financial outcomes after a booking change:

- surcharge,
- refund, if applicable,
- no price difference.

04. Shared Module for Participant Count Change Calculation

After validation and availability checks, the system must calculate the price difference caused by a change in the number of participants.

The same calculation concept shall apply to both Rosotravel products and Viator products.

04.1. Group Tours

For group tours priced per person, the difference shall be calculated as:

difference = (new participants – old participants) × price per person

The result shall be interpreted as follows:

- greater than zero → surcharge,
- less than zero → refund,
- equal to zero → no payment change.

04.2. Private Tours

For private tours with tier-based pricing by participant range, the price depends on participant tiers and not only on the exact number of participants.

The system must therefore:

- determine the old price based on the originally purchased participant count and pricing tier,
- determine the new price based on the new participant count and pricing tier,
- calculate the difference as:

difference = new price – old price

The result shall be interpreted as follows:

- greater than zero → surcharge,
- less than zero → refund,
- equal to zero → no payment change.

05. Notification Rule

After every successful booking change:

- the customer must receive a change confirmation and a new voucher,
- the booking status must be updated in the customer account,
- the system must send an email to **bookings@rosotravel.com** with change details and the voucher, subject to the rules defined below.

e. Booking Change Flow for Rosotravel Own Products

01. General Rule

For Rosotravel products, all decisions related to booking changes shall be based exclusively on:

- Rosotravel change and cancellation policy,
- Rosotravel booking statuses,
- Rosotravel availability.

No requests shall be sent to Viator for Rosotravel products.

02. Customer Request Submission

The customer may request changes to:

- participant count,
- date,
- time,
- language,
- pick-up,
- drop-off,
- participant name and traveler data.

03. Blocking Option Change Requests

If the customer attempts to change the tour option, the system must block the action and display the following message:

“Tour option cannot be changed. To choose a different option, cancel the booking and book again.”

04. Mandatory Validation

The back-end must validate:

- whether the current booking status allows changes,
- whether the requested change is allowed by Rosotravel policy,
- whether the requested change type is supported for the specific product,
- whether elements such as pick-up or drop-off are applicable.

05. Result of Negative Validation

If the requested change is not allowed, the system must:

- show a clear reason to the customer,
- optionally display a cancellation CTA if cancellation is allowed,
- end the process without payment,
- end the process without generating a voucher,
- not send an email to **bookings@rosotravel.com**.

06. Availability Check

If the change passes validation, the system must check availability for the new parameters, including:

- date,
- time,
- language,
- participant count,
- operational availability and logic for pick-up and drop-off, if applicable.

07. Price Difference Calculation

If the requested change affects price, including participant count changes, the system must calculate the price difference using the shared calculation module described above.

08. Surcharge, Refund, or Zero Difference

The system must support the following paths:

- if there is a surcharge, the customer pays and the change is then finalized,
- if there is a refund, the customer confirms and the change is finalized, with refund processing where applicable,

- if there is no price difference, the system finalizes the change without payment.

09. Finalization of the Change

If the change is completed successfully, the system must:

- store a new booking version,
- generate a new voucher.

010. Notifications on Success

After a successful booking change for a Rosotravel product:

- the customer must receive a change confirmation and a new voucher by email,
- the booking status must be updated in the customer account,
- the system must send an email to **bookings@rosotravel.com** containing change details and the voucher.

011. Cases Where bookings@rosotravel.com Must Not Be Notified

The system must not send an email to **bookings@rosotravel.com** in the following cases:

- the requested change is systemically impossible due to policy, status, or availability,
- the process was not finalized, for example because the customer did not complete payment.

f. Booking Change Flow for Viator Affiliate Products

01. General Rule

For Viator products, Rosotravel provides the user interface and handles payment where applicable, but the final decision regarding the booking change is made by Viator and the supplier.

02. Customer Request Submission

The customer may request changes to:

- participant count,
- date,
- time,
- language, if applicable,
- pick-up,

- drop-off, if applicable,
- participant name and traveler data, as required.

03. Mandatory Validation

Rosotravel back-end must:

- perform a local validation to determine whether the requested change appears possible,
- check status and known constraints,
- always send a check or quote request to Viator, because Viator has the final decision-making authority.

If Rosotravel already knows locally that the requested change is impossible due to a hard block, the system may stop the flow earlier and display an appropriate customer-facing message.

04. Sending the Change Request to Viator

The system must send to Viator:

- Rosotravel booking reference,
- requested change details,
- new date,
- new time,
- updated pick-up or drop-off details,
- traveler data,
- participant count,
- and any other relevant changed parameters.

05. Viator Response Handling

Viator may return:

- confirmation that the change is possible,
- confirmation that the change is free of charge,
- confirmation that the change requires a surcharge,
- information that the change is unavailable,
- an alternative proposal,
- a **manual follow-up required** status.

06. Free Change or Surcharge

If Viator confirms that the change is possible and requires a surcharge:

- Rosotravel must show the surcharge to the customer,
- the customer must pay on Rosotravel,

- after payment, Rosotravel must send the final change confirmation to Viator.

If Viator confirms that the change is possible and free of charge:

- Rosotravel must confirm the change in Viator immediately.

06.1. Participant Count Calculation Logic

Rosotravel may calculate the participant-count difference using the shared calculation module.

06.2. Binding Viator Amount

If Viator returns its own surcharge amount as a condition of the change, the Viator amount shall be binding.

07. Supplier Step

After Rosotravel confirms the change with Viator, Viator may forward the change to the supplier.

The supplier may:

- confirm the change,
- reject the change,
- propose an alternative.

08. Fetching Final Booking State

After the change process is completed, Rosotravel must:

- fetch the final booking details from Viator,
- store a new booking version in the Rosotravel database,
- generate a new voucher.

09. Notifications on Success

After a successful booking change for a Viator product:

- the customer must receive a change confirmation and voucher by email,
- the booking status must be updated in the customer account,
- the system must send an email to **bookings@rosotravel.com** containing change details and the voucher.

If the requested change is systemically impossible, the system must not notify **bookings@rosotravel.com**.

10. Notifications on Failure

If the booking change process fails, the following rules shall apply.

10.1. No Notification for Systemically Impossible Changes

If the requested change is systemically impossible, the system must not send an email to **bookings@rosotravel.com**.

10.2. Exception Cases Requiring Notification

The system must send an email to **bookings@rosotravel.com** if:

- the supplier actively rejected the request and the rejection was not automatic,
- Viator returned an alternative,
- the process ended with **manual follow-up required**,
- the customer did not confirm the surcharge or requested change within the allowed time window.

IV. Front-end - change for Viator products

1. Front-End Design Changes

01. General Front-End Design Direction

In accordance with the AI & SEO Scope, the objective is for developer to prepare front-end design change proposals for the key page templates and core page-level elements across the Rosotravel website, and then implement the approved changes.

This part of the scope concerns the visual, structural, and layout-related redesign of the main page types used within the Rosotravel website. The purpose of these works is to align the website with the new AI & SEO direction, improve consistency across page templates, and introduce an updated design approach across the most important website sections.

02. Front-End Design Changes

As part of this scope, Developer should prepare proposals for front-end changes, and after approval, implement the agreed changes to the main elements of the following page types and templates on the Rosotravel website:

- Homepage
- Country Page

- Region Page / Island / Other Entity Page
- City Page
- Things-to-Do Page Template (City-Level)
- Attraction Page
- Product Page
- Near-Page Template (City-Level)
- Blog Page
- Rosotravel Expert / Guide / Blogger Page
- Package / Expert Bundle Page
- Explore Attraction Map / AI Feed Context

03. Nature of the Planned Changes

The planned work in this section is intended to cover developer's preparation of front-end design proposals and the implementation of approved changes relating to the core elements of the above page types, including in particular:

- page layout,
- visual hierarchy,
- content block structure,
- positioning of key sections,
- presentation of content,
- navigation logic within the page,

04. Removal of the Previous Shop Page Model

An important implementation requirement is that the previous Shop page model, including pages such as: <https://rosotravel.com/tours/United-Kingdom/shop> shall be removed from the Rosotravel website structure.

The objective is to eliminate the separate Shop subpage as a required step in the purchase flow.

The developer shall implement the new page structure and purchase flow in such a way that the booking and purchase process can take place directly within the relevant target subpages and page templates, without the need to redirect the user to a separate Shop page.

Product presentation, selection logic, and purchase-related actions should therefore be integrated into the appropriate pages within the new website structure, in line with the AI & SEO Scope and the updated front-end design approach.

05. Purchase Flow Integration Within Target Pages

The new front-end approach shall ensure that the user is able to browse relevant products, review key offer information, and proceed toward booking directly from the appropriate page types within the Rosotravel website structure.

The developer shall implement the necessary front-end solutions so that the purchase path is embedded within the relevant destination, attraction, product, or related content pages, depending on the final page logic adopted under the AI & SEO Scope.

This approach is intended to replace the previous model in which product purchase required navigation to a separate Shop page.

2. General Front-End Information

01. Overall Goals for Front-End

The front-end implementation shall support the new Rosotravel website structure and user journey in line with the AI & SEO Scope, while also ensuring that all Viator-related requirements are properly reflected in the customer-facing experience.

The overall front-end objective is to combine updated design, improved usability, correct booking flow handling, and full operational alignment with the Viator integration model.

02. Compliance with Partner API Requirements

All functionalities implemented within the front-end scope must be designed and developed in accordance with the requirements and guidelines of the Viator Partner API.

This applies in particular to all customer-facing booking-related elements, partner content presentation, status communication, and user flow logic connected with Viator products. - [Viator Partner AP](#)

03. Meeting Viator Certification Requirements

One of the primary objectives of the front-end development is to fully meet all certification points agreed upon with Viator. -

 [Viator-API-Certification-front-end-checks.pdf](#)

All relevant front-end functionalities covered by the Viator certification process must be implemented in accordance with the agreed certification requirements and expected partner behavior.

The developer bears full responsibility for ensuring that the front-end part of the Viator affiliate products meets all certification requirements and successfully obtains Viator certification.

04. Completion Criteria

Front-end work shall be considered complete only once:

- all agreed certification points have been implemented, and
- the Viator certification process has been successfully passed.

3. Booking Confirmation Types – Global Concept

01. General Application

The booking confirmation type logic shall apply globally wherever booking status, booking behavior, or booking outcome is shown or communicated within the Rosotravel website and related customer-facing flows.

This includes, at minimum:

- product page,
- cart,
- checkout,
- emails,
- My Bookings area,

- booking summaries,
- booking-related notifications.

02. Instant Confirmation

Instant confirmation shall remain the current default booking method on the website for eligible products.

Under this model:

- products are automatically confirmed at the time of booking,
- the customer is charged immediately,
- the default main CTA label shall be “**Book now**”.

03. Manual Confirmation

Manual confirmation shall apply to products that require supplier approval before the booking can be confirmed.

Under this model:

- after the request is submitted, the booking remains in pending status until the supplier confirms or rejects it,
- supplier response is generally expected within up to 72 hours,
- the customer is not charged until confirmation is received, or alternatively only a pre-authorization may be placed depending on the final payment implementation,
- the default main CTA label shall be “**Send request**”.

04. Instant-Then-Manual Confirmation

Instant-then-manual confirmation shall apply to products that behave as instantly confirmable until a defined cutoff time before the tour starts, and then require supplier confirmation after that cutoff.

Under this model:

- before the cutoff time, the product behaves as an instant confirmation product,
- after the cutoff time, the product behaves as a manual confirmation product.

The CTA logic shall therefore be:

- before cutoff: “**Book now**”
- after cutoff: “**Send request**”

05. Front-End Display Requirement

The correct booking confirmation type must be clearly reflected in all relevant front-end elements so that the customer understands whether:

- the booking is confirmed immediately,
- the booking requires supplier confirmation,
- the booking behavior depends on the booking time relative to the cutoff.

4. Changes to Facilities, Labels, Tags, and Filters for Viator Affiliate Products

a. General Principle

01. Scope of Changes

The sections related to **Facilities**, **Labels**, **Tags**, and **Filters** for Viator affiliate products must be fully revised.

These elements should no longer be treated as fixed in their current form. They are to be changed in accordance with the new project approach and the updated target logic to be prepared by the developer.

02. Category Section

The **category section** has already been described earlier in this document and is therefore not repeated here.

However, the logic related to:

- product tags,
- product labels,
- product facilities,
- product filters,
- front-end display logic connected with those elements,

must be considered subject to full redesign.

03. Implementation Principle

This section does not define how these elements should be rebuilt technically.

The purpose of this section is only to indicate which elements must be changed and treated as part of the redesign under the new project assumptions prepared by the developer.

b. Elements Requiring Full Revision

01. Facilities for Viator Affiliate Products

The current logic for facilities assigned to Viator affiliate products must be fully revised.

This applies in particular to facilities currently connected with:

- Duration,
- Tour Type,
- Guide Type,
- Tour Vehicle,
- Skip The Line Tickets for Priority Access,
- the “lightning icon” label,
- any related front-end labels derived from product data.

02. Labels and Front-End Product Marking

The current front-end label logic for Viator affiliate products must be fully changed.

This includes all labels and visual markers shown on the front-end on the basis of affiliate product data, including in particular:

- Tour Type labels,
- booking-related labels,
- instant booking / “lightning icon” labels,
- guide-related labels,
- transport / vehicle-related labels,

- priority-access related labels.

03. Tag Logic

The current tag logic must be fully redesigned.

This applies both to:

- the assignment of tags to Viator affiliate products,
- the interpretation of those tags in the Rosotravel system,
- the use of those tags in downstream logic, including labels, facilities, and filters.

04. Filter Logic

The current filter logic connected with Viator affiliate products must also be fully revised.

This includes:

- filter values,
- filter grouping,
- filter assignment rules,
- the relationship between tags and filters,
- the way filters are presented and used on the front-end.

c. Data Areas Covered by the Revision

01. Duration

The current logic related to **Duration** must be revised.

This includes the way duration values are retrieved, interpreted, structured, and displayed for Viator affiliate products.

02. Tour Type

The current logic related to **Tour Type** must be revised.

This includes the rules used for assigning and displaying product types such as:

- Group Tour,
- Private Tour,
- Tickets,
- and any other related product type classifications via Viator API.

03. Booking-Related Labeling

The logic related to booking-oriented labels, including the current “**Just buy and we instantly organise**” concept and the related visual marking such as the **lightning icon**, must be revised as part of the new product display concept.

04. Guide Type

The current logic related to **Guide Type** must be revised.

This includes:

- guide-related product attributes,
- language-based guide logic,
- distinction between live guide and audio guide,
- display conditions for guide-related labels and facilities.

05. Tour Vehicle

The current logic related to **Tour Vehicle** must be revised.

This includes all classification and display logic connected with:

- transport-based tours,
- walking tours,
- and any related facility or label assignment.

06. Skip The Line / Priority Access

The current logic related to **Skip The Line Tickets for Priority Access** must be revised.

This includes both the product-data interpretation side and the front-end presentation of such elements.

d. Monitoring and Processing Logic

01. Processing Logs and Monitoring

The current data-processing logic related to website-side classification and monitoring must also be revised.

This applies in particular to:

- logs,
- metrics,
- number of changes,
- processing time,
- retry count,
- number of 429 errors,
- number of 503 errors,
- queue backlog.

02. Scope of Revision

These monitoring and processing-related sections should not remain in their current form and must be updated in line with the new project assumptions and the final solution prepared by the developer.

e. Final Scope Note

01. Summary of What Must Be Changed

For Viator affiliate products, the following elements are subject to full revision:

- Facilities,

- Labels,
- Tags,
- Filters,
- Duration logic,
- Tour Type logic,
- booking-related visual labels,
- Guide Type logic,
- Tour Vehicle logic,
- Skip The Line / Priority Access logic,
- website-side processing logs and monitoring metrics.

02. Design Basis

All of the above elements are to be changed in accordance with the new project approach and the updated solution to be prepared by the developer.

This section does not define the final implementation method, but it clearly establishes that the above areas are no longer to be maintained in their current form and must be rebuilt under the new project structure.

4. Global Payment Behavior

01. General Rule

The payment behavior displayed and applied within the cart and payment flow must correspond to the applicable booking confirmation type.

02. Instant Confirmation Payment Behavior

For instant confirmation products:

- payment shall be charged immediately,
- payment shall be captured instantly at the time of booking.

03. Manual Confirmation Payment Behavior

For manual confirmation products:

- the customer shall not be charged immediately, or
- an optional pre-authorization may be placed depending on the final implementation.

If pre-authorization is used, it may expire or be released if the booking is rejected or if the confirmation window times out.

04. Instant-Then-Manual Payment Behavior

For instant-then-manual confirmation products, the payment logic shall follow either the instant or manual model depending on whether the booking is made before or after the applicable cutoff time.

5. Global Rules for Partner Content

Important: This section may be changed if the AI & SEO Scope provides otherwise, as the rules regarding the marking, attribution, and presentation of Viator-related elements are still under discussion and may continue to be defined during the course of the project.

01. General Principle

Wherever the Rosotravel website displays content delivered by Viator, including but not limited to:

- viatorUniqueContent,
- reviews,
- supplier images,
- review photos,
- partner-provided descriptive blocks,

the front-end must handle such content in accordance with Viator's partner content requirements.

02. Clear Attribution for Partner Content

Wherever Viator requires attribution for partner-provided content, the front-end shall display clear and visible attribution identifying the source of such content.

Examples of acceptable attribution wording include:

- “**Content © provided by our partner Viator**”
- “**Customer reviews (from our partner Viator)**”

03. Explicit Partner Name Visibility

Where attribution is required, the name **Viator** or the relevant partner name must be explicitly visible within the relevant section.

This requirement applies in particular to:

- reviews,
- unique partner content blocks,
- partner-sourced images where required,
- customer review sections,
- any other area where partner-owned or partner-distributed content is displayed.

04. Front-End Consistency Requirement

The attribution and partner-content labeling logic must be implemented consistently across all relevant Rosotravel front-end templates and components where Viator content appears.

6. Pricing and Availability Rules for Affiliate Products

a. General Pricing Principle

01. Default Display Price

For all Viator affiliate products, the Rosotravel system shall use **price.original.recommendedRetailPrice** as the default displayed price.

This price shall be used on:

- listing pages,
- Homepage,
- Country Page,
- City Page,
- Attraction Page,
- Shop-related legacy contexts where still visible during transition,

- Product Page before the customer selects a date,
- all areas where a “**from price**” is shown.

02. Fallback Price Category Order

As a default rule, the system shall use the value for the **Adult** category. If the **Adult** category is not available, the system shall use the next available category in the following order:

- **ADULT**
- **SENIOR**
- **YOUTH**
- **CHILD**
- **INFANT**

03. Purpose of the Default Price Logic

The purpose of this rule is to ensure that the default displayed price is based on **Viator’s recommended public retail price**, which is the safest base price before an availability check is performed.

b. Pricing Rules Across the Website

01. General Rule

Viator may return different prices depending on:

- product option,
- date or season,
- age category,
- pricing model.

For this reason, the system must apply clear rules for selecting one displayed price for the user in each relevant context.

02. Listing Pages

On listing pages, including but not limited to:

- Homepage,
- Country Page,
- City Page,
- Attraction Page,
- and other listing-based page types,

the system shall display the lowest available **recommendedRetailPrice** for the **Adult** category across all product options.

If the **Adult** category does not exist, the system shall use the next available category in the following order:

- **Senior**
- **Youth**
- **Child**
- **Infant**

If the product has multiple options, the system shall select the lowest relevant price across all available product options and seasons.

The display format should follow the logic:

“From X PLN”

where **X** is the lowest selected **recommendedRetailPrice**.

03. Product Page Before Date Selection

On the Product Page, before the customer selects a date, the **“from price”** logic shall follow the same rule as on listing pages.

The Product Page shall therefore display the lowest relevant **recommendedRetailPrice** according to the same pricing hierarchy.

Once the customer selects a date and option, the displayed price must be automatically updated based on the result of the availability check.

04. Cart Pricing Rules

At the moment the product is added to the cart, the system must perform an availability and price check.

The price shown in the cart must then be replaced with:

price.bookable.retailPrice

If the cart price differs from the price previously shown on the Product Page, the cart must display a clear message informing the customer that the price has changed based on current availability.

The message should follow the logic:

“The price has been updated based on current availability.”

The cart must never use the default pre-check price as the final cart price.

05. Real-Time Price Update Before Payment

Before the customer proceeds to payment, the system must perform an availability check covering at minimum:

- selected date,
- selected option,
- number of travellers,
- selected age categories.

If the price returned by availability differs from the previously displayed price, the system must:

- overwrite the previous price with the current price,
- display a message informing the customer about the price change,
- block further payment progress until the customer accepts the updated price.

The customer-facing message should follow the logic:

“The price of this tour has changed. We have updated the amount payable.”

This rule is one of the key Viator certification requirements and must be treated as mandatory.

06. Final Price Used for Booking Submission

The final price sent in the booking request to Viator must always come from:

price.bookable.retailPrice

as returned during the availability check or booking check.

This rule is intended to ensure transactional consistency and to eliminate pricing differences between Rosotravel and Viator, including situations involving promotions on either the Viator side or the Rosotravel side.

c. Back-End Availability Rules for Affiliate Products

01. Product Type Identification

Each product must have a clearly defined product type.

The system must distinguish at minimum between:

- **ROSOTRAVEL** – internal product
- **AFFILIATE** – partner product, including products sourced through Viator

02. Availability Rules for Affiliate Products

For products marked as **AFFILIATE**, the internal Rosotravel availability system must remain inactive.

This means that affiliate products must not use Rosotravel's own internal availability management, including:

- internal calendars,
- internal time slots,
- internal seat or capacity limits,
- manual availability control on the Rosotravel side.

03. Admin Panel Rules

Within the admin panel, all availability-related elements for affiliate products must be:

- hidden, or
- blocked as read-only,

depending on the final implementation approach.

The administrator must not be able to manually manage or override affiliate product availability using Rosotravel's internal availability tools.

04. Single Source of Truth

For affiliate products, the only source of truth for availability-related and price-related data shall be the partner API, in particular the Viator API.

This applies in particular to:

- available dates,
- available times,
- number of available places, if returned by the partner,
- pricing returned together with availability,
- booking confirmation logic, including instant, manual, or mixed confirmation behavior.

05. Real-Time Availability Verification

Any customer action that may affect availability or final price must trigger a renewed availability verification through the partner API.

This applies in particular to:

- change of date,
- change of time,
- change of number of participants,
- change of age categories, including Senior,
- change of product option,

- change of language,
- change of pickup point, meeting point, or end point, where such elements affect the availability of the given tour.

06. Cache Rules

A short-term cache may be used only as an optional technical solution intended to reduce the number of API calls.

However:

- the cache must never replace or override partner data,
- in the event of any discrepancy, the response returned by the partner API shall always prevail.

d. Tour Page Application

01. Scope of Application

The above rules shall apply to the Tour Page and to all related booking-flow elements where affiliate-product pricing, availability, or booking-confirmation behavior is presented, checked, recalculated, or revalidated.

7. Searches

01. Overall goals for front-end

The search bar is a global component, visible and accessible at the top of:

- Homepage
- City pages
- Country pages
- Attraction pages
- Product pages

Design:

- Same height, width, and style across the site (desktop + mobile).

1. Search capabilities

The search must support multiple input types with smart suggestions:

- Supported search types
 - Category – e.g. “Walking tour”, “Food & Wine”.
 - Attraction – landmarks or POIs (e.g. “Colosseum”).
 - Free text / keywords – arbitrary text (“Harry Potter”, “museum night”).
 - Product code / ID – internal or partner product references (“LON1234”).
 - City / country – existing behavior (“Paris”, “Rome”, etc.).

Important: If a user types the name of a country or city into the search bar and confirms the search (e.g., by pressing Enter or clicking the search icon), the system must automatically redirect them to the corresponding dedicated page the country page or city page instead of showing the general Shop search results.

a. Behaviour based on input type

- If the user types only a category (e.g. “Walking tour”):
 - First, the system should try to determine the user’s current destination if location services are enabled.
 - If the user has previously selected a destination, the system should interpret the category within that destination by default.
- If the user searches by attraction, country, or city:
 - The search engine compares the input with attraction and destination data coming from the backend.

Title & Code

Description & Highlights

Location

Tags - sections on city page

Included & Excluded

Cancellation Policy

Before Booking & Voucher Info

Photos

Tour Options

Facilities

Reviews

SEO and Translate Tour

Debug info:
current step: 3
product_id: 243
country_id: 44
city_id: 47

You are editing the trip: 5 Top Churches in Munich Private Guided Walking Tour

Select country from where tour starts

Germany

Select city from where tour starts

Munich

Type attraction which customer will visit during their trip (all attractions names in English)

Enter a location

Marienplatz Odeonsplatz Frauenkirche Mariensäule New Town Hall St. Peter Asamkirche

St. Michael München Theatine Church

Save tour Continue

- If the user enters a product code (ID):
 - The exact matching product must appear first.
 - Then, additional related products (similar attraction / destination / product family) may be shown below.

b. Handling typos and fuzzy searching

The system must support typo-tolerance and fuzzy matching:

- If the user enters a misspelled destination, category, or attraction, the search should still recognise the likely intended term.
- When a typo is detected, the search bar should display suggested corrected keywords along with relevant results.

Examples:

- “Barcelna” → “Barcelona”
- “musum night” → “museum night”

This improves usability and ensures users can always find what they are looking for, even with imperfect input.

Below I’m adding a link that illustrates how the destination-search flow is intended to work, including the required search capabilities and supported input types. -

<https://www.awesomescreenshot.com/video/46584057?key=4ad3560ecebdc33b7e38759fb104658>

- Smart suggestions (auto-complete)
 - As user types, show dropdown suggestions mixing:
 - cities,
 - attractions,
 - categories,
 - products / product codes.
 - Each suggestion has a small icon/label indicating the type (City, Attraction, Category).
 - Results ranked by relevance and popularity.
- Data sources:
 - Rosotravel database (titles, tags, descriptions, attractions, product names),
 - Here you’ll find a link where you can review the full content of a single product -
<https://www.awesomescreenshot.com/video/46633492?key=f6890de447c69e2fafbd5e849681b2fa>
- Behavior
 - Typing triggers immediate suggestions.
 - Pressing Enter or selecting a suggestion opens a dedicated search results page (Shop section).

8. Tour page / Product detail

a. New booking types & facilities icons

On the product page, near the availability/booking section, show booking type icon & description:

Instant confirmation (⚡)

- Icon: ⚡
- Label: “Instant confirmation”

- Short description: “Just buy and we instantly organize. Your booking ensures the tour is already fully organized – just come and enjoy!”

Manual confirmation (⏳)

- Icon: ⏳
- Label: “On request”
- Short description: “Request and wait for confirmation. This tour operates only when the supplier confirms availability, usually within 72 hours. No payment is charged until confirmed.”

Instant-then-manual (⚡ ⏳)

- Icon: ⚡ ⏳
- Label: “Instant until [X h] before start, then on request”
- Short description: “The tour is confirmed instantly if you book early enough. Closer to the start time, the supplier must confirm availability manually.”

CTA label logic:

- Before cutoff (instant) → “Book now”.
- After cutoff / manual → “Send request”.

b. Reviews for affiliate products (**This section can be changed by SEO & AI Scope**)

Placement

- Tour operator / supplier information

Layout

- Title:

★ Customer reviews (from our partner [PartnerName])

- Each review card:
 - Avatar/photo (if provided),
 - Name and date,
 - Star rating,
 - Review text exactly as received,
 - Optional customer photos in lightbox gallery.
- Footer:
 - Content © provided by our partner [PartnerName]
 - Partner logo
 - Note such as: “Reviews displayed as originally submitted by customers on [PartnerName].”
 - Link: “See all reviews on [PartnerName]” with rel="nofollow noopener noreferrer".

Technical rules

- No editing, paraphrasing, or translating reviews.
- Preserve original language and punctuation.
- Reviews and review images must not be indexable by search engines (e.g. wrap in <!-- noindex --> or equivalent).
- Styling must match Rosotravel's design while keeping partner attribution clearly visible.

c. Images (supplier & customer)

Supplier images from partner API:

- Display as-is, no cropping/overlay or watermark modifications.
- Must not be indexable by search engines.

Customer review images from API:

- Also unmodified and unindexed.

Layout can reuse current Rosotravel gallery UI (carousel/lightbox), but content must remain original.

8.1 Availability section ("Check Availability")

a. Real-time availability validation (Viator API) – REQUIRED

The system must continuously validate the availability and pricing of the selected product option using Viator's Availability API in real time.

Validation must be triggered automatically when the user changes ANY of the following parameters:

- Date
- Time / Time slot
- Number of travelers
- Traveler types (Adult / Youth / Child / Infant / Senior)
- Selected language
- Selected option/variant

For each change, the system must:

- Send an availability check request to Viator.
- Immediately update:
 - available timeslots,
 - final price per traveler type,
 - option availability,
 - language availability,
 - cutoff rules (affecting CTA logic).

If availability becomes invalid:

- show an error message (e.g., “This option is no longer available for the selected date”),
- force the user to select an alternative date/time before proceeding.

b. Age Categories

New age category: Senior

Add Senior as a standard age band alongside:

- Adult
- Youth
- Child
- Infant

Age ranges must be defined entirely in the backend (e.g., Senior = 65+). All age bands must be sent to Viator exactly as required per product/option.

c. Language Options — With Type Labels

For each available language, the system must display an additional Language Type Tag:

- Live – in-person guided
- Written – written / self-guided materials
- Audio – audio-guided materials

These tags must appear:

- In the availability section, next to the selectable language.
- In the selected option details, once the user chooses an option.

User explanation tooltip

Next to each tag (Live / Written / Audio) show a small info icon ().

On hover (desktop) or tap (mobile), display a short tooltip:

- Live: “Guided live by a professional guide.”
- Written: “Includes written materials to follow the route independently.”
- Audio: “Includes an audio guide to follow the tour at your own pace.”

Tooltips must be fully customizable in the backend (translatable).

d. Tour Option Display (Variants & Guide Information)

In the list of available variants/options:

- Display the chosen language + the Language Type Tag.
- Ensure the tag is always consistent with the language selected in the Availability section.

- If the option supports multiple languages, show all supported tags.

Example option row:

“Italian — Live (icon)”

e. Itinerary (Viator products only)

01. Purpose

Some affiliate products — specifically those provided by Viator — include a detailed tour itinerary describing the step-by-step flow of the experience. To ensure transparency and consistency with the original offer, the product page must support a dedicated Itinerary section.

This section does not exist on the current Rosotravel product pages and must be introduced only for Viator products.

02. Display rule

- The Itinerary section must be displayed exclusively for products with source = VIATOR.
- For Rosotravel products and for partner products that do not provide itinerary data:
 - the section must not be rendered at all,
 - no empty state, placeholder, or fallback should appear.

03. Placement on the product page

The Itinerary section should be positioned:

- below the main tour description, and
- above operational / legal sections, such as:
 - Important Information
 - Cancellation Policy
 - Tour Operator / Supplier Information

04. Itinerary content

The section presents the detailed tour program exactly as provided by Viator.

Content characteristics:

- step-by-step itinerary items (stops / stages),
- stop or location name,
- description of each stop,
- duration per stop (if available),
- labels such as “Stop at” or “Pass by” (if provided).
- pictures (if provided)

The itinerary must reflect the original Viator structure and wording.

05. Front-end logic

IF product.source == VIATOR AND itinerary data exists

→ render Itinerary section

ELSE

→ do not render the section

06. Example of itinerary from Viator

Itinerary.

- 1 The Rembrandt Hotel**
Meet your guide in front of The Rembrandt Hotel, 11 Thurloe Pl, South Kensington, London SW7 2RS, United Kingdom. Please do not enter the hotel, it is only a meeting point and the staff ... [Read more](#)
10 minutes • Admission Ticket Free
- 2 V&A - Victoria and Albert Museum**
Private tour of the Victoria and Albert Museum to immerse yourself in art history with an Expert Guide. It's a museum where everyone will find something that they like, from antiquitie ... [Read more](#)
1 hour 20 minutes • Admission Ticket Free
- 3 South Kensington**
Enhance your V&A experience with the highlights of South Kensington. The Victoria and Albert Museum is located in a very affluent district, home to many museums, prestigious ven ... [Read more](#)
1 hour • Admission Ticket Free
- 4 London**
Book a 1,5-hour round-trip transfer and a 2-hour private tour of the rich collection of the Victoria and Albert Museum. Private transfers include both pickup and drop-off at your accom ... [Read more](#)
1 hour • Admission Ticket Free
- 5 Royal Albert Hall**
We also offer a group tour of the Victoria and Albert Museum and South Kensington borough. Book a 4-hour group tour to visit the V&A Museum, discover its incredible collection of art a ... [Read more](#)
1 hour • Admission Ticket Free

f. Cancellation policy & voucher type

Cancellation policy pop-up

- Must show the exact policy text from the partner API.
- Add note: "The cancellation policy shown below is provided directly by the supplier."

Voucher type annotation

- If voucher type differs from Rosotravel's standard:
 - Add a line at the end of "Important information", e.g.:

- “Paper voucher required”,
- “Printed ticket only”,
- “Mobile ticket accepted”.

g. Tour operator / supplier information

Placement

- Directly under “Important information”.

Main label

- Text: “Supplied by [Supplier Name]” – clickable, opens a modal with full supplier details.

Modal content (if provided via API)

- Trading name
- Legal name
- Address
- Phone
- Email (public contact)
- Business registration number
- Registered in

Mandatory notes at bottom of modal (for Viator products)

- Self-certification statement, e.g.: “This operator has self-certified that they are committed to only offer services that comply with applicable laws, including EU law.”
- Viator marketplace disclosure, e.g.: “This supplier is a trader on the Viator marketplace. Viator is an online intermediation platform that allows consumers to buy experiences from third-party providers. For your protection, make sure you use Viator's platform to communicate with operators and pay for bookings.”

These texts must be displayed exactly as provided by Viator for Viator-affiliated products.

h. Pricing and Availability Rules for Affiliate Products

9. Changes to Discount Codes with Affiliation Information

a. Selectable List for the Discount Codes Section

01. Product Scope Dropdown

Within the “**Discount codes**” section, there shall be a dropdown list defining to which product source a given code applies.

Field: **Applies to / Product source**

Type: **dropdown**

Available options:

- **Rosotravel** (default)
- **Viator**
- **All (Rosotravel + Viator)**

02. Purpose of the Field

This field shall determine whether the discount code applies only to Rosotravel products, only to Viator products, or to both product sources.

b. Discount Code Logic in the Cart

01. Product Source Verification

For each cart item, the system shall verify the **product source** assigned to that item:

- **Rosotravel**
- **Viator**

The system shall then compare the product source of each cart item with the scope assigned to the discount code.

02. Discount Application Rules

If the code is set to **Rosotravel**, the discount shall be applied only to Rosotravel products.

If the code is set to **Viator**, the discount shall be applied only to Viator products, and only after confirmation through the affiliate partner endpoint.

If the code is set to **All**, the following logic shall apply:

- for Rosotravel products, the discount shall be calculated locally,
- for Viator products, the discount shall be calculated only after verification through the affiliate partner endpoint.

03. Products Outside the Scope of the Code

Products that do not fall within the scope assigned to the discount code shall not be covered by the discount.

c. Cart Communication to the User

01. Applied Code Information

The cart shall display clear information to the user indicating which items the discount code has actually been applied to.

This information shall be presented in the form of a short list containing the titles of the tours for which the discount was successfully applied.

02. Message Format

The message should follow the format:

“Codes have been applied for:

- Tour title 1
- Tour title 2”

03. Display Rule

Only those items for which the discount has actually been applied shall be displayed in this list.

Items not included in the discount calculation shall not be listed.

10. Cart page

a. Main Purpose of the Cart

The Cart Page is a single unified page where the user must be able to:

- View all tours added to the booking
- Complete all required traveler information and booking questions
- Provide pickup / meeting location details (if required)
- Choose cancellation policy (only for Rosotravel products)
- Review price behavior (what is charged now vs later)
- Proceed to the payment step

The entire cart must remain fully synchronized with Viator API requirements, especially booking questions, confirmation types, and pickup logic.

General scheme of cart page:

```

| CART PAGE |
|
| [Main CTA – ALWAYS visible, fixed top-right] |
| - "Go to required tour details" OR "Proceed to payment" |
|
|

```

TOUR #1

```

| Tour title € Price |
| Date / Time |
| Status: ✓ Completed or ! Details required |
|
| [ Fill details ] |
|

```

TOUR #2

```

| Tour title € Price |
| Date / Time |
| Status: ! Details required |
|
| [ Fill details ] (opens panel) |
|

```

...more tours

b. Display of Tours in Cart

- All tours added to the booking must appear in the cart as separate blocks.
- Each tour shows:
 - Title
 - Date
 - Price
 - Confirmation type (⚡ instant / ⌚ on request / ⚡⌚ mixed)
 - Status of details:
 - "Details required"
 - "All details completed"

01. Expandable detail sections (per tour)

- Each tour includes a button such as "Fill in details".

- Clicking this button opens ONLY that tour's detail form.
- Only one tour detail section can be expanded at a time.
- When a section is expanded → all other sections collapse.

02. Completed tours

If all required fields are already filled:

- The tour displays:
 "All required information has been completed."
- The detail form remains collapsible for edits.

General details panel for complete for one tour, details to fill out are depends from bookings:

TOUR DETAILS – expanded **only for THIS** tour

1. LEADER CUSTOMER

First name: [.....]

Last name : [.....]

Email : [.....]

Phone : [+.....]

2. TRAVELLER DETAILS

For each participant:

- **First** name
- **Last** name
- Age band
- Weight / Height (if required)
- Special requirements

3. PICKUP OPTIONS

() Provide pickup now

[**search** box **with** suggestions + icons]

() Provide later

() Meet **at** departure point

4. **END** POINT (if required)

[**search** box **with** suggestions]

5. CONDITIONAL QUESTIONS (if **any**)

- Cruise ship name (if SEA)
- Arrival/Departure mode

[Save details **for** this tour]

c. CTA Behavior Based on Missing Data

- If any product in the cart still requires details:

- The main CTA becomes:
“Go to missing tour details”
 - Clicking it auto-scrolls or opens the nearest tour requiring details.
- If all details are complete, the CTA becomes:
“Order now”

d. Persistence of User Input

To protect user progress:

- All entered data must be cached temporarily, so if:
 - user leaves the cart,
 - the browser crashes,
 - the website reloads,

The cart automatically restores all previously entered values (only within a reasonable time window).

e. Booking Questions — Full Integration with Viator API

Each product may have different required booking questions.

The Cart must display exactly the fields required by Viator for that specific product/option.

01. Types of booking questions the system must support (Examples from Viator API)

- Traveler First Name (FULL_NAMES_FIRST)
- Traveler Last Name (FULL_NAMES_LAST)
- AGE_BAND (Adult, Youth, Child, Infant, Senior)
- Weight (WEIGHT)
- Height (HEIGHT)
- SPECIAL_REQUIREMENTS
- CONTACT methods (email, phone)
- Pickup location (LOCATION_REFERENCE or FREETEXT)
- Transfer-related questions (arrival, departure modes)
- Cruise ship / port-required questions
- Any conditional booking questions

Important: The Backend must map every Viator booking question to a UI input.

02. Leader Customer Rules (Mandatory)

Each cart booking must include:

- Leader: First name
- Leader: Last name
- Leader: Email
- Leader: Phone number
 - Must satisfy Viator rules:

- Must begin with “+” + country code
- Must contain sufficient digits
- Allowed characters: digits, hyphens, spaces

Other travelers do not require separate phone numbers.

f. Pickup, Meeting Point, End Point (Viator-compliant)

01. If the tour supports PICKUP

Show **three options**:

- Provide pickup location now
- Provide pickup details later
- Meet at the departure point

01.1 Provide pickup now

Search bar must retrieve locations from:

- Viator Location Reference API
- Rosotravel internal location DB
- Google Maps (only when FREETEXT is allowed)

Returned suggestion icons:

-  Airport
-  Cruise port
-  Train station
-  Hotel
-  Other POI

Mapping:

```
PICKUP_POINT = {
  unit: LOCATION_REFERENCE or FREETEXT,
  answer: "<selected_value>"
}
```

01.2. Provide later

Maps to:

```
PICKUP_POINT = {
  unit: LOCATION_REFERENCE,
  answer: "CONTACT_SUPPLIER_LATER"
}
```

01.3. Meet at departure point

Maps to:

```
PICKUP_POINT = {
  unit: LOCATION_REFERENCE,
  answer: "MEET_AT_DEPARTURE_POINT"
}
```

General Scheme for meeting point/pick-up logic:



02. If the tour DOES NOT support pickup

- Do NOT show pickup UI
- Display only the meeting point from:
logistics.start[] (Viator API)
- Internally send default values (e.g., OTHER) if Viator requires them

03. END POINT (tour may require a drop-off location)

If the product requires an end point:

- Display End Point search identical to pickup
- Location sources identical
- Mapping identical:

```
END_POINT = {  
  unit: LOCATION_REFERENCE or FREETEXT,  
  answer: "<selected_value>"  
}
```

g. NEW MANDATORY RULE (Viator-flexible booking questions)

If Viator allows skipping a field → our system must also allow skipping it

If Viator marks a field as optional or deferrable:

- The cart must allow the user to proceed without filling it
- The missing field must not block checkout
- The item must be labeled:
“This information can be provided later in Your Bookings.”
- After booking, the user must be able to add the missing data inside the My Bookings panel.

This ensures full alignment with Viator certification behavior.

h. Additional Tour Information Requirements

01. Guide Type

Cart must display the selected guide type:

- Live
- Written
- Audio

These values come from Viator product options.

i. Cancellation Policy Section (Rosotravel ONLY)

01. Visibility rule (VERY IMPORTANT)

The “Choose cancellation policy” section:

- Appears ONLY if all products in the cart are Rosotravel products
- There is one price calculated for each product like it is already on the website

If at least one product in the cart is not a Rosotravel product (e.g. partner/Viator product):

- The entire cancellation policy section must NOT be displayed at all
- No Basic/Premium choice is available in the cart

Choose cancellation policy:

☒	BASIC	 We care about your comfort
Cancellation up to 24 hours before the tour starts		FREE
Cancel for any reason and get a full refund of your money in 7 days.		
Terms and Conditions		
☐	PREMIUM	 Perfect solution for people traveling by plane
Cancellation up to 24 hours before the tour starts		
Cancel for any reason and get a full refund of your money in 2 days.		
Emergency cancellation up to 2 hours before the tour starts		25,50 €
In case of positive Covid test, plane delay or accident and get a full refund in 2 days.		
Terms and Conditions		

[ADD MORE TOURS](#)

Total price
204,00 €

The current Premium cancellation policy calculation logic works exactly as it does now (no changes required).

11. Payment page & Post-booking

No Viator requirement to show “Powered by Viator” or similar here. The important part is **status and communication** for instant vs on-request items.

a. Final summary & actions

Show a clear breakdown:

- all tours and their confirmation types,
- total price,
- amount charged now,
- amount to be charged later (if applicable).

Main action button:

- Instant-only bookings → “Confirm and pay”.
- Mixed/On-request → wording consistent with the fact that some items will stay in “request” status; charges must reflect Viator workflow.

Optional checkbox (business logic; not Viator-mandated but allowed):

- “Charge me automatically once my booking is confirmed (total: €YYY).”

b. Post-checkout communication (Viator requirements)

As a merchant partner, Rosotravel is responsible for sending all:

- booking confirmations,
- amendments,
- cancellation notifications

to the customer (Viator provides updates via API / email, but we are the customer-facing party).

We must therefore:

- Send confirmation emails for instant items immediately.
- Send “request received / pending confirmation” emails for on-request items, stating the expected time window (e.g. up to 72h).
- Send follow-up emails when:
 - booking is confirmed by supplier (via Viator API updates),
 - booking is rejected or expires,
 - booking is cancelled (with refund info as per Viator’s cancellation result).

“My bookings” area (if implemented) must reflect:

- Confirmed bookings (voucher/ticket available),
- Pending bookings (with clear indication of pending status and time),
- Rejected/expired bookings (option to choose alternative).]